

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
LẬP TRÌNH THIẾT BỊ DI ĐỘNG

ĐỀ TÀI: QUẢN LÝ ĐẶT XE CÔNG NGHỆ

Nhóm sinh viên thực hiện:

1. Cao Xuân Điệp – 6451071019 – CQ.64.CNTT
2. Lương Thị Kim Ngân – 6451071049 – CQ.64.CNTT
3. Phạm Hải Yến – 6451071091 – CQ.64.CNTT

Tp. Hồ Chí Minh - 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
LẬP TRÌNH THIẾT BỊ DI ĐỘNG

ĐỀ TÀI: QUẢN LÝ ĐẶT XE CÔNG NGHỆ

Nhóm sinh viên thực hiện:

1. Cao Xuân Điệp – 6451071019 – CQ.64.CNTT
2. Lương Thị Kim Ngân – 6451071049 – CQ.64.CNTT
3. Phạm Hải Yến – 6451071091 – CQ.64.CNTT

Giảng viên hướng dẫn: Th.S Nguyễn Thiện Dương

Tp. Hồ Chí Minh - 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TpHCM, ngày ... tháng ... năm 2026

Giảng viên hướng dẫn

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời tri ân chân thành đến Ban Giám hiệu và quý Thầy/Cô Trường Đại học Giao thông Vận tải Phân hiệu tại thành phố Hồ Chí Minh nói chung, cũng như các Thầy/Cô trong Bộ môn Công nghệ Thông tin nói riêng. Những kiến thức quý báu được truyền đạt trong suốt quá trình học tập tại trường chính là nền tảng vững chắc để nhóm có thể thực hiện và hoàn thành đề tài này.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc nhất đến thầy ThS. Nguyễn Thiện Dương. Trong suốt thời gian thực hiện dự án “Quản lý đặt xe công nghệ”, thầy đã luôn tận tình hướng dẫn, chỉ bảo và đưa ra những định hướng đúng đắn giúp nhóm giải quyết các vấn đề khó khăn. Sự quan tâm và những lời khuyên của thầy không chỉ giúp sản phẩm được hoàn thiện về mặt kỹ thuật mà còn giúp các thành viên học hỏi thêm nhiều kinh nghiệm quý báu trong tư duy giải quyết vấn đề.

Nhóm thực hiện cũng xin gửi lời cảm ơn đến gia đình, bạn bè đã luôn ở bên động viên, khích lệ và hỗ trợ trong suốt quá trình làm đồ án.

Mặc dù đã nỗ lực hết mình, nhưng do thời gian và kiến thức còn hạn chế nên đề tài khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp ý kiến, nhận xét từ quý Thầy/Cô để hệ thống được hoàn thiện hơn và để nhóm có thể rút ra những bài học kinh nghiệm cho công việc sau này.

Chúng em xin chân thành cảm ơn!

TpHCM, ngày ... tháng ... năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	i
LỜI CẢM ƠN	ii
MỤC LỤC	iii
DANH MỤC CHỮ VIẾT TẮT	vii
DANH MỤC HÌNH ẢNH	ix
DANH MỤC BẢNG BIỂU	xi
CHƯƠNG 1. MỞ ĐẦU	1
1.1 Giới thiệu đề tài	1
1.2 Mục tiêu đề tài	2
1.3 Mục đích nghiên cứu	3
1.4 Đối tượng nghiên cứu	3
1.5 Phạm vi nghiên cứu	3
1.6 Phương pháp nghiên cứu	4
1.7 Ý nghĩa thực tiễn của đề tài	4
1.8 Cấu trúc cuốn báo cáo đồ án	5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	7
2.1 Tổng quan về lập trình ứng dụng di động	7
2.2 Flutter và ngôn ngữ Dart	8
2.2.1 Tổng quan về Flutter	8
2.2.2 Ngôn ngữ Dart	9
2.3 Kiến trúc Clean Architecture trong Flutter	9
2.4 ASP.NET Core Web API và SignalR	11

2.4.1 ASP.NET Core Web API.....	11
2.4.2 SignalR	12
2.5 Xác thực và phân quyền với JWT.....	12
2.6 Cơ sở dữ liệu SQL Server	13
2.7 Nền tảng bản đồ: OpenStreetMap, Nominatim và OpenRouteService	13
2.7.1 OpenStreetMap và flutter_map	13
2.7.2 Nominatim API.....	14
2.7.3 OpenRouteService	15
2.8 Khảo sát các ứng dụng đặt xe tiêu biểu	15
2.8.1 Ứng dụng Grab	15
2.8.2 Ứng dụng Be.....	16
2.8.3 Nhận xét và định hướng đề tài.....	17
2.9 Kết luận và lựa chọn công nghệ cho đề tài	17
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG	19
3.1 Mô tả bài toán	19
3.2 Phân tích yêu cầu hệ thống	20
3.2.1 Yêu cầu chức năng.....	20
3.2.2 Yêu cầu phi chức năng	21
3.3 Thiết kế giao diện người dùng	22
3.3.1 Giao diện Khách hàng	22
3.3.2 Giao diện Tài xế	23
3.3.3 Giao diện Quản trị viên	24
3.4 Phân tích và thiết kế bằng UML	25
3.4.1 Sơ đồ Use Case	25

3.4.2 Sơ đồ BPMN (Business Process Model and Notation)	28
3.4.3 Sơ đồ Sequence (Tuần tự)	28
3.4.4 Sơ đồ Class	30
3.5 Thiết kế cơ sở dữ liệu.....	31
3.5.1 Mô hình thực thể liên kết (ERD)	31
3.5.2 Thiết kế các bảng dữ liệu chính.....	32
3.6 Thiết kế kiến trúc hệ thống	33
CHƯƠNG 4: CÀI ĐẶT THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	37
4.1 Môi trường phát triển.....	37
4.1.1 Công cụ và phần mềm	37
4.1.2 Cấu trúc dự án.....	38
4.2 Cài đặt các chức năng chính	39
4.2.1 Chức năng xác thực và đăng nhập.....	39
4.2.2 Chức năng đặt xe và hiển thị bản đồ	40
4.2.3 Chức năng theo dõi chuyển đi realtime	42
4.2.4 Chức năng phân hệ Tài xế	43
4.2.5 Chức năng đánh giá và lịch sử chuyến đi.....	45
4.2.6 Chức năng Quản trị viên.....	47
4.3 Sử dụng công cụ AI Antigravity trong phát triển hệ thống.....	47
4.3.1 Giới thiệu Antigravity.....	47
4.3.2 Ứng dụng AI trong từng giai đoạn phát triển	48
4.3.3 Đánh giá hiệu quả sử dụng AI	49
4.4 Thực nghiệm và kiểm thử hệ thống	49
4.4.1 Môi trường kiểm thử	49

4.4.2 Kết quả kiểm thử chức năng.....	50
4.4.3 Kiểm thử giao diện và tương thích thiết bị.....	51
4.5 Đánh giá hệ thống	51
4.5.1 Mức độ hoàn thành chức năng	51
4.5.2 Ưu điểm và hạn chế	52
KẾT LUẬN	54
5.1 Kết quả đạt được	54
5.1.1 Về mặt lý thuyết	54
5.1.2 Về mặt thực nghiệm	54
5.2 Hạn chế còn tồn đọng	55
5.3 Hướng phát triển trong tương lai	56
TÀI LIỆU THAM KHẢO.....	57

DANH MỤC CHỮ VIẾT TẮT

STT	Từ viết tắt	Ý nghĩa
1	3NF	Dạng chuẩn 3 trong chuẩn hóa cơ sở dữ liệu
2	AOT	Ahead-of-Time (Biên dịch trước)
3	APK	Android Package Kit
4	AVD	Android Virtual Device (Máy ảo Android)
5	BFD	Sơ đồ phân cấp chức năng
6	BPMN	Business Process Model and Notation (Sơ đồ quy trình nghiệp vụ)
7	DFD	Sơ đồ luồng dữ liệu
8	EF	Entity Framework
9	ERD	Mô hình thực thể liên kết
10	FCM	Firebase Cloud Messaging
11	FK	Khóa ngoại (Foreign Key)
12	IDE	Môi trường phát triển tích hợp (Ví dụ: VS Code, Visual Studio)
13	JIT	Just-in-Time (Biên dịch tại thời điểm chạy)
14	JWT	JSON Web Token
15	KH	Khách hàng
16	ODbL	Giấy phép dữ liệu mở (Open Database License)
17	ORM	Object-Relational Mapper
18	ORS	OpenRouteService
19	OSM	OpenStreetMap

20	PK	Khóa chính (Primary Key)
21	RDBMS	Hệ quản trị cơ sở dữ liệu quan hệ
22	SRS	Tài liệu yêu cầu hệ thống
23	SSMS	SQL Server Management Studio
24	T-SQL	Ngôn ngữ truy vấn T-SQL
25	TX	Tài xế
26	UC	Use Case
27	UML	Ngôn ngữ mô hình hóa thống nhất (Dùng để vẽ các sơ đồ hệ thống)

DANH MỤC HÌNH ẢNH

Hình 1.1 Tăng trưởng thị trường ứng dụng đặt xe công nghệ tại Việt Nam.....	1
Hình 1.2 Kiến trúc tổng thể của hệ thống LeafGo	5
Hình 2.1 So sánh kiến trúc phát triển ứng dụng Native và Cross-platform.....	8
Hình 2.2 Kiến trúc ba tầng của Flutter.....	9
Hình 2.3 Mô hình Clean Architecture.....	10
Hình 2.4 Cấu trúc thư mục Flutter project LeafGo theo Clean Architecture.....	11
Hình 2.5 Luồng giao tiếp giữa Flutter client, REST API và SignalR Hub	12
Hình 2.6 SQL Server và Entity Framework Core trong hệ sinh thái .NET	13
Hình 2.7 Hiển thị bản đồ OpenStreetMap qua flutter_map	14
Hình 2.8 Giao diện đặt xe trên ứng dụng Grab	16
Hình 2.9 Giao diện đặt xe trên ứng dụng Be.....	17
Hình 3.1 Màn hình đặt xe của khách hàng trong ứng dụng LeafGo.....	22
Hình 3.2 Màn hình theo dõi chuyến đi và đánh giá tài xế sau chuyến	23
Hình 3.3 Giao diện nhận chuyến và cập nhật trạng thái chuyến đi của tài xế	24
Hình 3.4 Giao diện bảng điều khiển của Quản trị viên trong ứng dụng LeafGo.....	25
Hình 3.5 Sơ đồ Use Case tổng quát của hệ thống LeafGo.....	26
Hình 3.6 Sơ đồ Use Case chi tiết cho UC02 (Đặt xe).....	27
Hình 3.7 Sơ đồ Use Case chi tiết cho UC07 (Nhận chuyến)	27
Hình 3.8 Sơ đồ BPMN mô tả luồng xử lý quy trình đặt xe	28
Hình 3.9 Sơ đồ BPMN mô tả luồng xử lý quy trình hoàn thành chuyến.....	28
Hình 3.10 Sơ đồ Sequence mô tả luồng đặt xe từ.....	29
Hình 3.11 Sơ đồ Sequence khi tài xế cập nhật trạng thái.....	30
Hình 3.12 Sơ đồ Class các lớp nghiệp vụ chính của hệ thống LeafGo.....	31

Hình 3.13 Sơ đồ thực thể liên kết (ERD) của cơ sở dữ liệu LeafGo	32
Hình 3.14 Kiến trúc tổng thể hệ thống LeafGo.....	35
Hình 4.1 Cấu trúc thư mục dự án Flutter và backend ASP.NET Core	39
Hình 4.2 Màn hình đăng ký và đăng nhập của ứng dụng LeafGo	40
Hình 4.3 Màn hình đặt xe hiển thị lộ trình trên nền bản đồ	41
Hình 4.4 Màn hình chờ taxi nhận chuyển và thông tin chuyến đi	42
Hình 4.5 Màn hình theo dõi vị trí taxi theo thời gian thực trên bản đồ.....	43
Hình 4.6 Thông báo chuyến mới và màn hình xác nhận nhận chuyến của taxi.....	44
Hình 4.7 Màn hình taxi cập nhật trạng thái chuyến đi theo từng bước.....	45
Hình 4.8 Màn hình đánh giá taxi và lịch sử chuyến đi.....	46
Hình 4.9 Màn hình Dashboard và Quản lý taxi của Quản trị viên.....	47
Hình 4.10 Giao diện Antigravity AI hỗ trợ sinh mã nguồn Flutter trong VS Code..	48

DANH MỤC BẢNG BIỂU

Bảng 3.1 Danh sách yêu cầu chức năng hệ thống LeafGo	20
Bảng 3.2 Mô tả các trạng thái của chuyến đi (Trip Status).....	33
Bảng 4.1 Môi trường và công cụ phát triển hệ thống LeafGo	37
Bảng 4.2 Kết quả kiểm thử chức năng hệ thống LeafGo.....	50
Bảng 4.3 Tổng hợp mức độ hoàn thành các chức năng hệ thống	52
Bảng 5.1 Tổng hợp kết quả đạt được của đề tài LeafGo.....	55

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	Nội dung công việc	Nhận xét của trưởng nhóm	Phần trăm đóng góp (%)
1	Phạm Hải Yến (<i>Nhóm trưởng</i>)	- Đảm nhiệm vai trò Nhóm trưởng: Lập kế hoạch, phân chia công việc, điều phối và giám sát tiến độ dự án. - Thiết kế và lập trình giao diện Frontend phân hệ Khách hàng (đặt xe, theo dõi tài xế) và phân hệ Quản trị viên (Admin Dashboard). - Thiết kế và vẽ các sơ đồ tương tác động hệ thống: Sơ đồ lớp (Class Diagram), Sơ đồ tuần tự (Sequence Diagram), Sơ đồ trạng thái (State Diagram). - Tham gia viết và tổng hợp báo cáo: Phụ trách Chương 1 (Mở đầu) và Chương 5 (Kết luận & Kiến nghị).	- Hoàn thành xuất sắc vai trò quản lý và điều phối nhóm, giữ vững tiến độ chung. - Giao diện Frontend thiết kế hiện đại, responsive mượt mà. - Các sơ đồ thiết kế UML logic, chuẩn xác và đúng chuẩn kỹ nghệ phần mềm.	33%
2	Lương Thị Kim Ngân	- Khảo sát bài toán thực tế (Grab), thu thập yêu cầu và phân tích đặc tả tài liệu yêu cầu hệ thống (SRS). - Thiết kế và lập trình giao diện Frontend phân hệ Tài xế (nhận chuyển, cập nhật trạng thái) và	- Làm việc nghiêm túc, có tính kỷ luật cao, luôn hoàn thành công việc đúng hạn. - Đặc tả yêu cầu chi tiết và hệ thống hóa sơ đồ nghiệp	33%

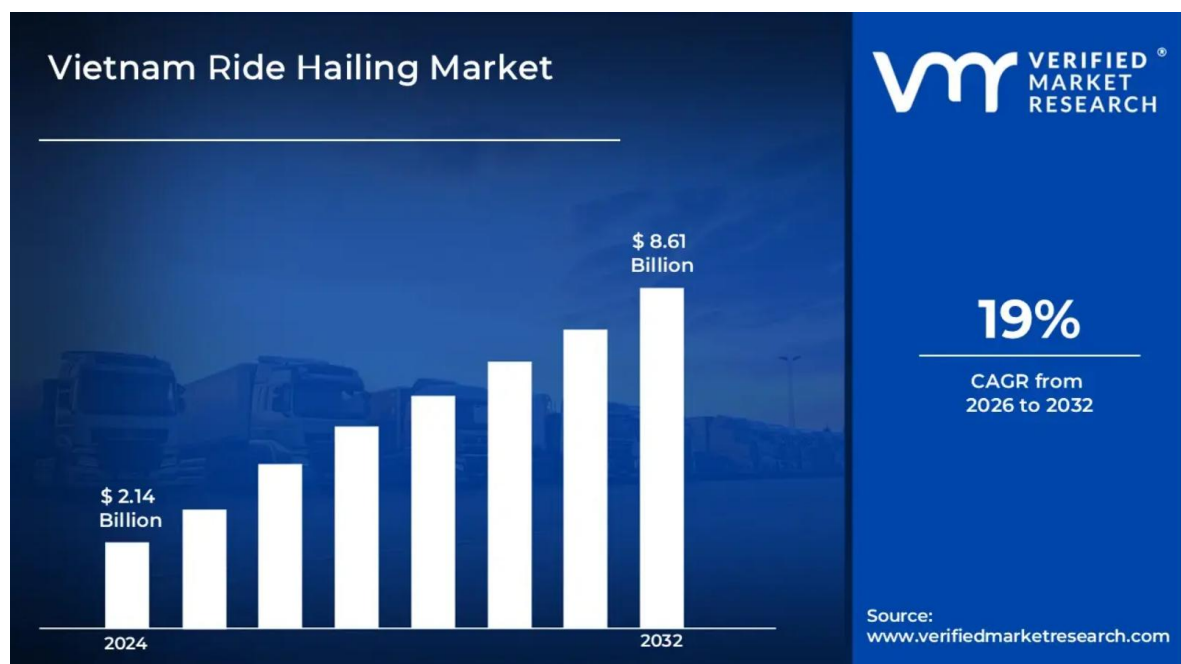
		phân hệ Xác thực (Đăng ký/Đăng nhập). - Thiết kế các sơ đồ quy trình nghiệp vụ hệ thống: Sơ đồ phân cấp chức năng (BFD), Sơ đồ luồng dữ liệu (DFD), Sơ đồ quy trình nghiệp vụ (BPMN), Sơ đồ ca sử dụng (Use Case Diagram). - Tham gia viết báo cáo: Phụ trách Chương 2 (Cơ sở lý thuyết) và Chương 4 (Cài đặt thực nghiệm - phần Kịch bản kiểm thử Test Case).	vụ rất chuẩn chỉ, chính xác. - Giao diện phân hệ Tài xế trực quan, dễ thao tác.	
3	Cao Xuân Điệp	- Thiết kế cấu trúc cơ sở dữ liệu quan hệ (vẽ sơ đồ ERD, lược đồ quan hệ, chuẩn hóa 3NF và xử lý phá chuẩn có kiểm soát). - Lập trình hệ thống Backend API (ASP.NET Core/Node.js) xử lý các luồng nghiệp vụ cốt lõi. - Cấu hình và phát triển cơ chế thời gian thực Websockets/SignalR phục vụ cho việc đồng bộ trạng thái chuyển đi giữa khách hàng và tài xế. - Tham gia viết báo cáo: Phụ trách Chương 3 (Thiết kế hệ thống - phần CSDL) và Chương 4 (Cài đặt thực nghiệm - phần Ứng dụng AI).	- Thành viên có năng lực kỹ thuật tốt, chủ động và có trách nhiệm cao. - Hệ thống API viết sạch, tối ưu hóa tốt cơ chế xử lý đồng thời (Optimistic Locking) chống tranh chấp tài xế. - CSDL được thiết kế chặt chẽ, tối ưu.	33%

CHƯƠNG 1. MỞ ĐẦU

Chương này trình bày tổng quan về đề tài “Ứng dụng quản lý đặt xe công nghệ LeafGo trên nền tảng Android sử dụng Flutter”. Nội dung bao gồm bối cảnh thực tiễn dẫn đến sự cần thiết của đề tài, mục tiêu và mục đích nghiên cứu, đối tượng và phạm vi triển khai, phương pháp tiếp cận, ý nghĩa thực tiễn, cũng như cấu trúc tổng thể của cuốn báo cáo. Từ đó tạo tiền đề để triển khai nền tảng lý thuyết và kỹ thuật ở các chương tiếp theo.

1.1 Giới thiệu đề tài

Trong bối cảnh cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, các dịch vụ giao thông thông minh đã và đang trở thành một phần không thể thiếu trong đời sống đô thị. Tại Việt Nam, sự bùng nổ của các nền tảng đặt xe công nghệ như Grab, Be hay Gojek đã thay đổi hoàn toàn thói quen di chuyển của hàng triệu người dùng, đặc biệt tại các thành phố lớn như Thành phố Hồ Chí Minh và Hà Nội. Người dùng ngày nay không chỉ cần một phương tiện di chuyển mà còn kỳ vọng vào trải nghiệm đặt xe nhanh chóng, minh bạch và đáng tin cậy ngay trên thiết bị di động.



Hình 1.1 Tăng trưởng thị trường ứng dụng đặt xe công nghệ tại Việt Nam

Tuy nhiên, hầu hết các ứng dụng hiện có đều thuộc sở hữu của các tập đoàn nước ngoài với mô hình vận hành phức tạp, chi phí hoa hồng cao và ít linh hoạt trong việc tùy chỉnh theo nhu cầu địa phương. Điều này tạo ra nhu cầu thực tiễn cho việc nghiên cứu và xây dựng một hệ thống đặt xe công nghệ có thể triển khai ở quy mô vừa và nhỏ, phù hợp với đặc thù của thị trường trong nước.

Xuất phát từ thực tiễn đó, nhóm đề xuất và thực hiện đề tài “Ứng dụng quản lý đặt xe công nghệ LeafGo” - một hệ thống đặt xe xây dựng trên nền tảng Android sử dụng Flutter, kết hợp với backend .NET, cơ sở dữ liệu SQL Server, dịch vụ Firebase và tích hợp bản đồ OpenStreetMap thông qua Mapbox. Hệ thống hướng đến việc kết nối ba nhóm người dùng chính gồm khách hàng, tài xế và quản trị viên trong một quy trình vận hành thống nhất, rõ ràng và có thể mở rộng.

1.2 Mục tiêu đề tài

Về mặt lý thuyết, đề tài hướng đến việc tìm hiểu và vận dụng các kiến thức về lập trình di động với Flutter và ngôn ngữ Dart, bao gồm kiến trúc ứng dụng theo mô hình MVVM, quản lý trạng thái và luồng dữ liệu trong ứng dụng Android. Bên cạnh đó, nhóm cũng nghiên cứu cách thiết kế và xây dựng RESTful API với .NET, tổ chức cơ sở dữ liệu quan hệ trên SQL Server, tích hợp Firebase Authentication cho việc xác thực người dùng, và sử dụng Mapbox SDK để hiển thị bản đồ cùng tính năng định vị theo thời gian thực.

Về mặt thực nghiệm, đề tài đặt mục tiêu xây dựng và hoàn thiện một ứng dụng Android có đầy đủ ba phân hệ: phân hệ khách hàng với các chức năng đặt xe, theo dõi chuyến đi và đánh giá dịch vụ; phân hệ tài xế với các chức năng nhận chuyến, cập nhật trạng thái và xem thông kê thu nhập; phân hệ quản trị viên với các chức năng quản lý tài khoản, giám sát hoạt động và điều chỉnh cài đặt hệ thống. Sản phẩm cần đảm bảo hoạt động ổn định, giao diện thân thiện và có khả năng mở rộng trong tương lai.

1.3 Mục đích nghiên cứu

Nghiên cứu này được thực hiện nhằm hai mục đích song song. Thứ nhất, về phía học thuật, đề tài giúp nhóm thực hành toàn bộ quy trình phát triển phần mềm di động - từ phân tích yêu cầu, thiết kế kiến trúc hệ thống, lập trình và kiểm thử cho đến đánh giá sản phẩm - qua đó củng cố và nâng cao năng lực kỹ thuật của từng thành viên trong môi trường làm việc nhóm thực tế.

Thứ hai, về phía ứng dụng, đề tài hướng đến việc tạo ra một giải pháp công nghệ có tính khả thi cao cho bài toán quản lý dịch vụ đặt xe quy mô nhỏ và vừa. Hệ thống LeafGo có thể được triển khai thực tế như một nền tảng kết nối tài xế cá nhân với khách hàng, đồng thời cung cấp công cụ quản lý và giám sát toàn diện cho đơn vị vận hành. Đây là hướng nghiên cứu có ý nghĩa thiết thực trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ tại Việt Nam.

1.4 Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài là quy trình nghiệp vụ của hệ thống đặt xe công nghệ, bao gồm cơ chế phối hợp giữa các tác nhân (khách hàng, tài xế, quản trị viên), luồng xử lý từ khi đặt xe đến khi hoàn thành chuyến đi, và các quy tắc nghiệp vụ liên quan đến định giá, phân công tài xế, đánh giá và xử lý vi phạm.

Bên cạnh đó, đề tài nghiên cứu các công nghệ được sử dụng trong quá trình triển khai gồm: framework Flutter và ngôn ngữ Dart cho lập trình giao diện Android, nền tảng .NET cho phát triển API backend, hệ quản trị cơ sở dữ liệu SQL Server, dịch vụ Firebase, và thư viện Mapbox Flutter GL cho hiển thị bản đồ và xử lý tọa độ địa lý. Đây đều là những công nghệ phổ biến trong ngành phát triển ứng dụng di động hiện đại và có cộng đồng hỗ trợ rộng lớn.

1.5 Phạm vi nghiên cứu

Về nền tảng, ứng dụng LeafGo được triển khai dành riêng cho hệ điều hành Android, không bao gồm phiên bản iOS hay ứng dụng web. Phiên bản Android tối thiểu được hỗ trợ là Android 8.0 (API level 26) trở lên, đảm bảo tương thích với phần lớn thiết bị Android đang lưu hành tại Việt Nam tính đến thời điểm thực hiện đề tài.

Về chức năng, đề tài tập trung vào các nghiệp vụ cốt lõi của một nền tảng đặt xe: đăng ký và xác thực tài khoản, đặt và theo dõi chuyến xe, nhận và xử lý chuyển đi của tài xế, quản lý người dùng và thống kê của quản trị viên. Các tính năng nâng cao như thanh toán trực tuyến, tích hợp tổng đài, hay chức năng gợi ý lộ trình thông minh dựa trên AI nằm ngoài phạm vi của đề tài này và có thể được xem xét trong các nghiên cứu tiếp theo.

Về dữ liệu và môi trường thử nghiệm, hệ thống được kiểm thử trên thiết bị thật và máy ảo Android với dữ liệu mô phỏng. Toàn bộ quá trình phát triển được thực hiện trong thời gian học kỳ với quy mô nhóm ba thành viên.

1.6 Phương pháp nghiên cứu

Để đạt được mục tiêu lý thuyết, nhóm tiến hành nghiên cứu tài liệu từ các nguồn chính thức như tài liệu Flutter SDK, tài liệu .NET và SQL Server của Microsoft, tài liệu Firebase của Google, và Mapbox Developer Documentation. Ngoài ra, nhóm tham khảo các bài báo khoa học, công trình liên quan và ứng dụng thực tế trên thị trường nhằm đúc kết kinh nghiệm thiết kế và kiến trúc phù hợp.

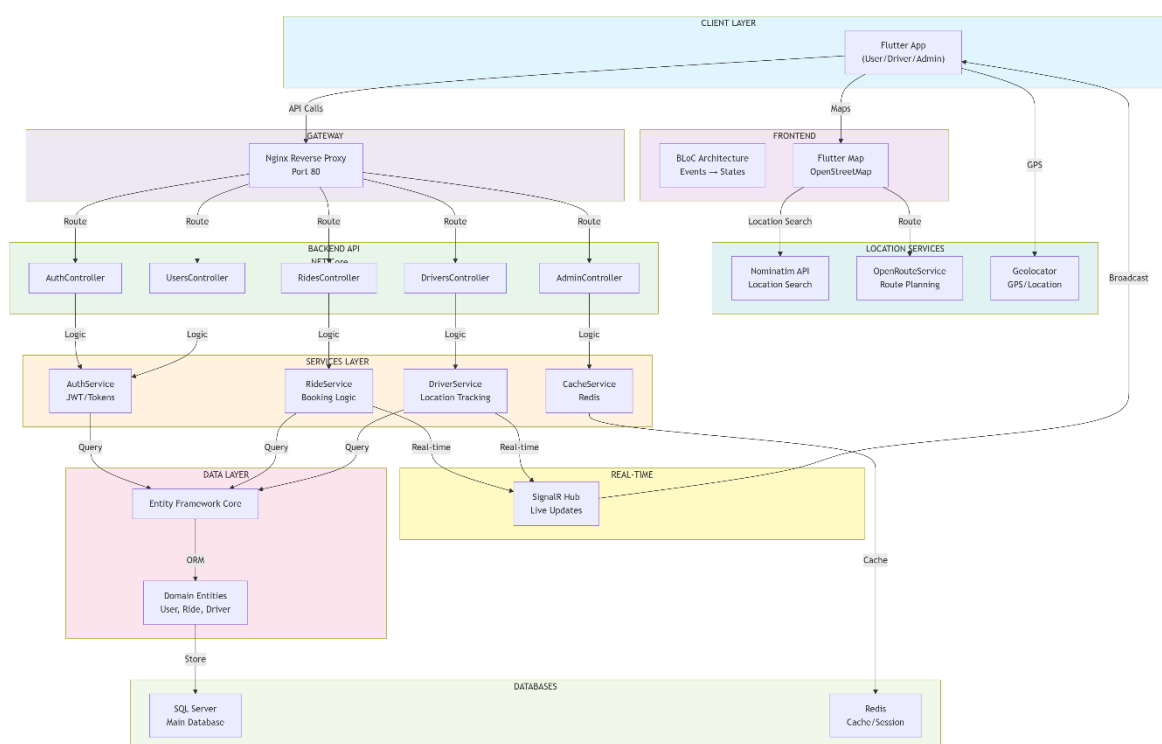
Để đạt được mục tiêu thực nghiệm, nhóm áp dụng phương pháp phát triển phần mềm theo mô hình lặp (iterative), trong đó mỗi vòng lặp hoàn thiện một tập chức năng và được kiểm thử ngay trước khi chuyển sang vòng tiếp theo. Quá trình thiết kế hệ thống sử dụng UML để mô hình hóa nghiệp vụ, và các test case được xây dựng theo phương pháp kiểm thử chức năng kết hợp kiểm thử biên. Bên cạnh đó, nhóm khai thác các công cụ hỗ trợ lập trình bằng AI như GitHub Copilot và Claude nhằm tăng tốc độ triển khai và nâng cao chất lượng mã nguồn.

1.7 Ý nghĩa thực tiễn của đề tài

Đối với người dùng cuối, ứng dụng LeafGo cung cấp trải nghiệm đặt xe trực quan và minh bạch ngay trên điện thoại Android. Khách hàng có thể theo dõi trạng thái chuyến đi theo thời gian thực thông qua bản đồ tích hợp, biết trước chi phí ước tính và lịch sử tất cả các chuyến đã đi. Về phía tài xế, ứng dụng giúp quản lý công việc hiệu quả hơn với thống kê thu nhập rõ ràng và giao diện nhận chuyến đơn giản.

Đối với đơn vị vận hành, hệ thống cung cấp bảng điều khiển quản trị cho phép theo dõi toàn bộ hoạt động, quản lý tài khoản và điều chỉnh chính sách một cách tập trung. Đây là nền tảng có thể áp dụng thực tế cho các doanh nghiệp vận tải quy mô nhỏ muốn số hóa hoạt động mà không cần đầu tư vào các nền tảng thương mại đắt tiền.

Về mặt học thuật và kỹ thuật, đề tài là minh chứng thực tế về khả năng xây dựng một hệ thống đa tầng hoàn chỉnh - từ giao diện di động Flutter đến API .NET và cơ sở dữ liệu SQL Server - trong khuôn khổ đồ án môn học, góp phần bổ sung vào kho tài liệu tham khảo cho các nghiên cứu tiếp theo trong lĩnh vực ứng dụng di động và dịch vụ giao thông thông minh.



Hình 1.2 Kiến trúc tổng thể của hệ thống LeafGo

1.8 Cấu trúc cuốn báo cáo đồ án

Báo cáo được tổ chức thành bốn chương chính với nội dung liên kết chặt chẽ.

Chương 1 - Mở đầu (chương này) giới thiệu tổng quan về đề tài, xác lập mục tiêu, phạm vi và phương pháp nghiên cứu, tạo nền tảng định hướng cho toàn bộ báo cáo.

Chương 2 - Cơ sở lý thuyết trình bày các kiến thức nền tảng liên quan đến đề tài, bao gồm tổng quan về lập trình di động với Flutter, kiến trúc ứng dụng và các công nghệ được sử dụng (.NET, SQL Server, Firebase, Mapbox), đồng thời khảo sát và đánh giá các ứng dụng đặt xe công nghệ đã có trên thị trường.

Chương 3 - Phân tích và thiết kế hệ thống đi sâu vào quá trình phân tích yêu cầu chức năng và phi chức năng, xây dựng các mô hình UML mô tả nghiệp vụ và tương tác, thiết kế cơ sở dữ liệu, kiến trúc hệ thống và giao diện người dùng.

Chương 4 - Cài đặt thực nghiệm và đánh giá trình bày chi tiết quá trình triển khai ứng dụng, mô tả cấu trúc source code, minh họa các chức năng chính qua giao diện thực tế, mô tả quá trình sử dụng công cụ AI hỗ trợ phát triển, kết quả kiểm thử và đánh giá toàn diện hệ thống.

Kết luận: Chương 1 đã xác lập rõ bối cảnh, mục tiêu và phạm vi của đề tài “Ứng dụng quản lý đặt xe công nghệ LeafGo”. Từ thực tiễn nhu cầu đặt xe công nghệ tại Việt Nam, nhóm xác định hướng xây dựng ứng dụng Android sử dụng Flutter kết hợp với backend .NET, SQL Server, Firebase và Mapbox. Những định hướng này là cơ sở để triển khai nghiên cứu cơ sở lý thuyết và kỹ thuật cụ thể trong chương tiếp theo.

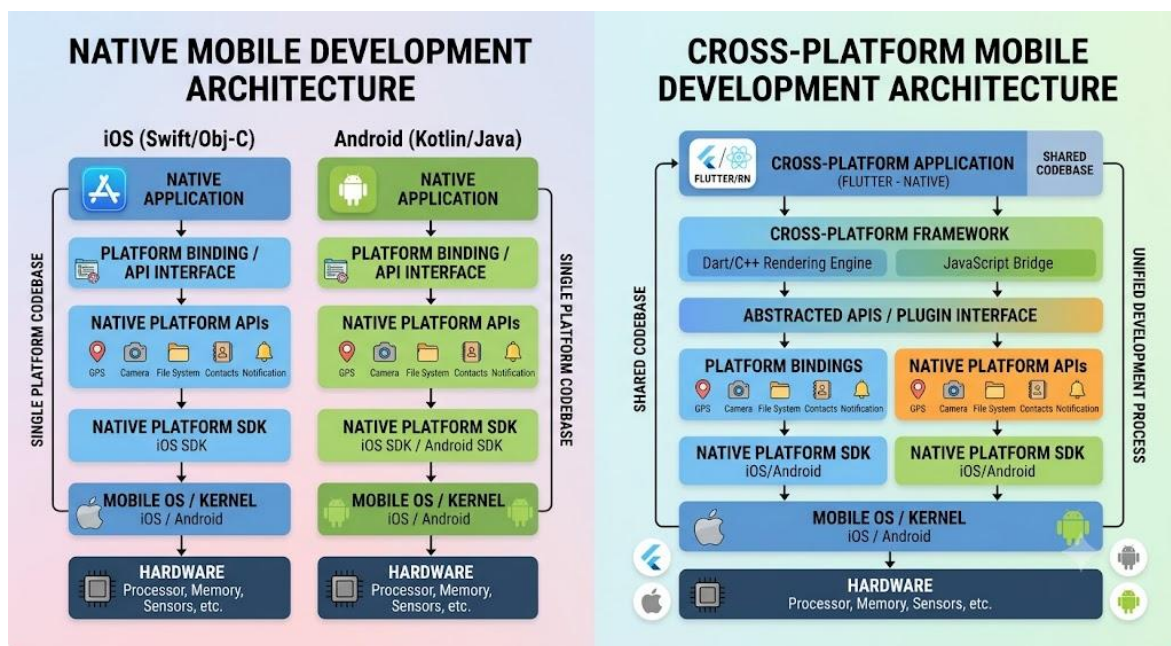
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng lý thuyết và công nghệ phục vụ cho quá trình xây dựng ứng dụng LeafGo. Nội dung bao gồm: tổng quan về lập trình di động và framework Flutter; kiến trúc Clean Architecture được áp dụng trong ứng dụng; các công nghệ backend (.NET, SignalR, JWT); hệ quản trị cơ sở dữ liệu SQL Server; dịch vụ xác thực Firebase Authentication; và nền tảng bản đồ mã nguồn mở OpenStreetMap kết hợp OpenRouteService. Bên cạnh đó, chương cũng phân tích các ứng dụng đặt xe tiêu biểu trên thị trường, từ đó xác định định hướng và đóng góp của đề tài.

2.1 Tổng quan về lập trình ứng dụng di động

Lập trình ứng dụng di động là lĩnh vực phát triển phần mềm tập trung vào việc xây dựng các ứng dụng chạy trên thiết bị di động như điện thoại thông minh và máy tính bảng. Có hai hướng tiếp cận chính: phát triển ứng dụng native - viết riêng cho từng hệ điều hành (Android bằng Kotlin/Java, iOS bằng Swift) - và phát triển đa nền tảng (cross-platform) cho phép dùng chung một codebase để triển khai trên nhiều hệ điều hành khác nhau. Hướng tiếp cận đa nền tảng ngày càng được ưa chuộng nhờ tiết kiệm chi phí và rút ngắn thời gian phát triển, trong khi vẫn đảm bảo hiệu năng và trải nghiệm người dùng gần với ứng dụng native.

Hiện nay, hai framework đa nền tảng phổ biến nhất là React Native (Meta) và Flutter (Google). Đề tài này lựa chọn Flutter vì những ưu điểm vượt trội phù hợp với yêu cầu của hệ thống LeafGo, được trình bày chi tiết tại mục 2.2. [1]

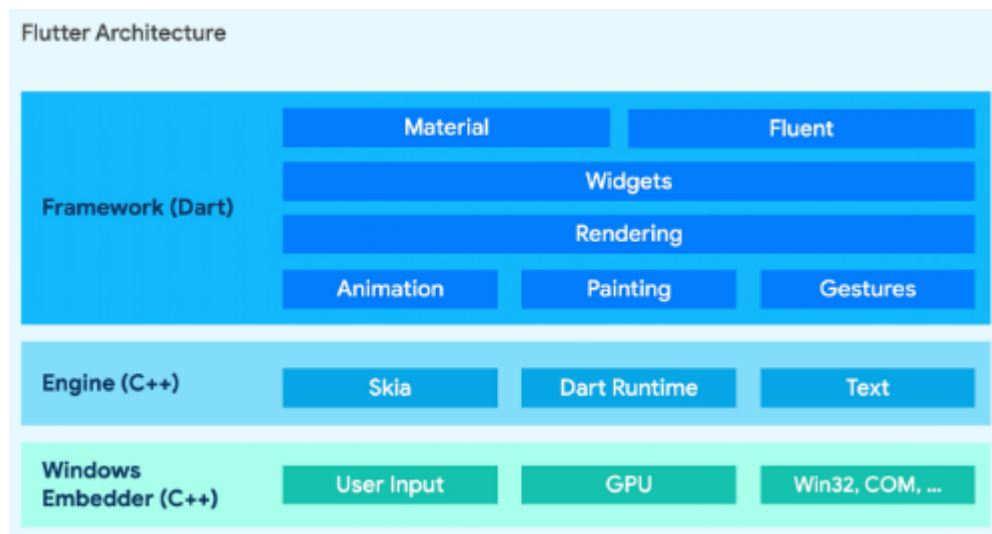


Hình 2.1 So sánh kiến trúc phát triển ứng dụng Native và Cross-platform

2.2 Flutter và ngôn ngữ Dart

2.2.1 Tổng quan về Flutter

Flutter là framework mã nguồn mở do Google phát triển, ra mắt phiên bản ổn định đầu tiên vào tháng 12 năm 2018. Flutter cho phép xây dựng ứng dụng đa nền tảng - Android, iOS, Web và Desktop - từ một codebase duy nhất viết bằng ngôn ngữ Dart. Điểm khác biệt cốt lõi của Flutter so với các framework cross-platform khác là không sử dụng các thành phần giao diện gốc của hệ điều hành (native widgets), thay vào đó tự vẽ toàn bộ giao diện thông qua công cụ đồ họa Skia (hoặc Impeller từ Flutter 3.x), giúp đảm bảo tính nhất quán hoàn toàn trên mọi thiết bị. [2]



Hình 2.2 Kiến trúc ba tầng của Flutter

Kiến trúc Flutter được tổ chức thành ba lớp chính. Lớp nền (Embedder) tương tác trực tiếp với hệ điều hành để xử lý vòng lặp sự kiện, render surface và truy cập API nền tảng. Lớp Engine chứa runtime Dart, công cụ vẽ đồ họa và các thư viện cốt lõi. Lớp Framework (viết hoàn toàn bằng Dart) cung cấp toàn bộ widget, hệ thống animation, routing và các tiện ích phát triển. Sự phân tầng rõ ràng này giúp Flutter đạt hiệu năng cao và dễ dàng mở rộng. [2]

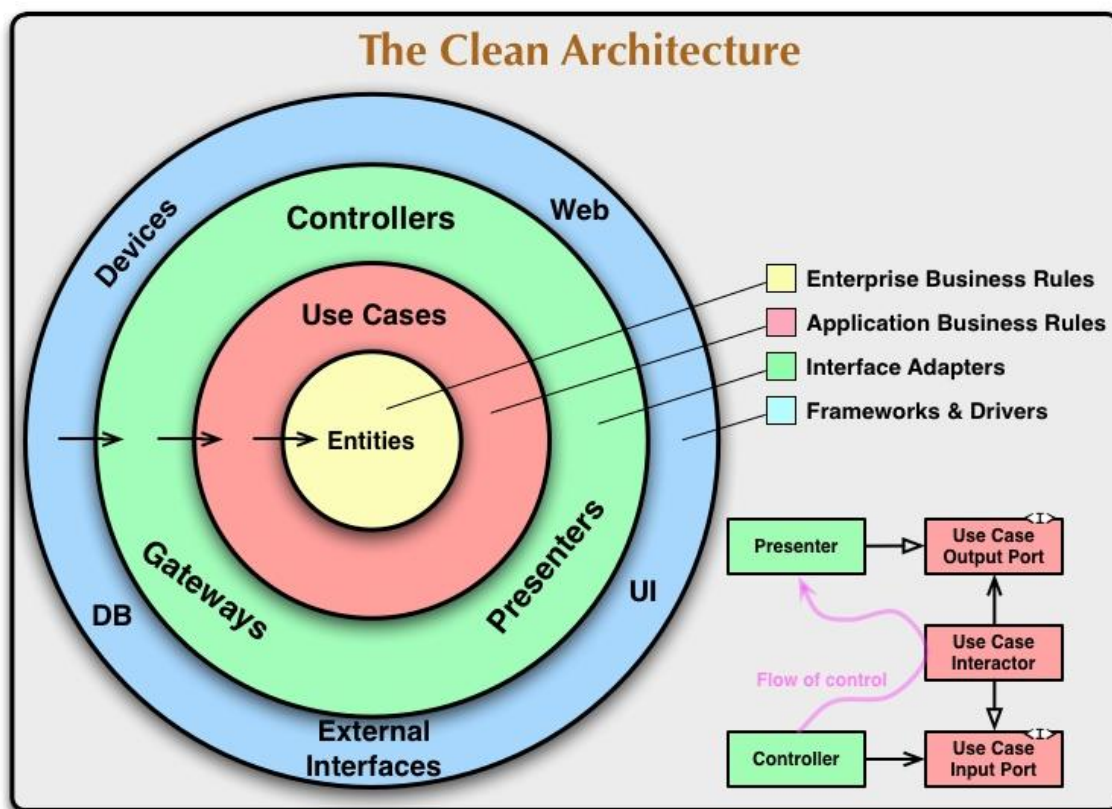
2.2.2 Ngôn ngữ Dart

Dart là ngôn ngữ lập trình hướng đối tượng, kiểu tĩnh (statically typed), do Google phát triển. Dart hỗ trợ lập trình bất đồng bộ thông qua cơ chế *async/await* và *Future*, rất phù hợp với các thao tác I/O trong ứng dụng di động như gọi API hay truy cập thiết bị. Dart được biên dịch theo hai chế độ: biên dịch AOT (Ahead-of-Time) khi build release để đạt hiệu năng tối ưu, và JIT (Just-in-Time) trong quá trình phát triển để hỗ trợ tính năng Hot Reload - cho phép thấy thay đổi giao diện ngay lập tức mà không cần khởi động lại ứng dụng. [3]

2.3 Kiến trúc Clean Architecture trong Flutter

Clean Architecture là nguyên lý kiến trúc phần mềm được Robert C. Martin (Uncle Bob) đề xuất, với mục tiêu tạo ra hệ thống độc lập với framework, cơ sở dữ liệu và giao diện người dùng thông qua việc phân tách rõ ràng các tầng trách nhiệm.

Nguyên tắc trung tâm là *Dependency Rule*: các tầng bên trong không được phép phụ thuộc vào tầng bên ngoài, chỉ có chiều ngược lại. [4]

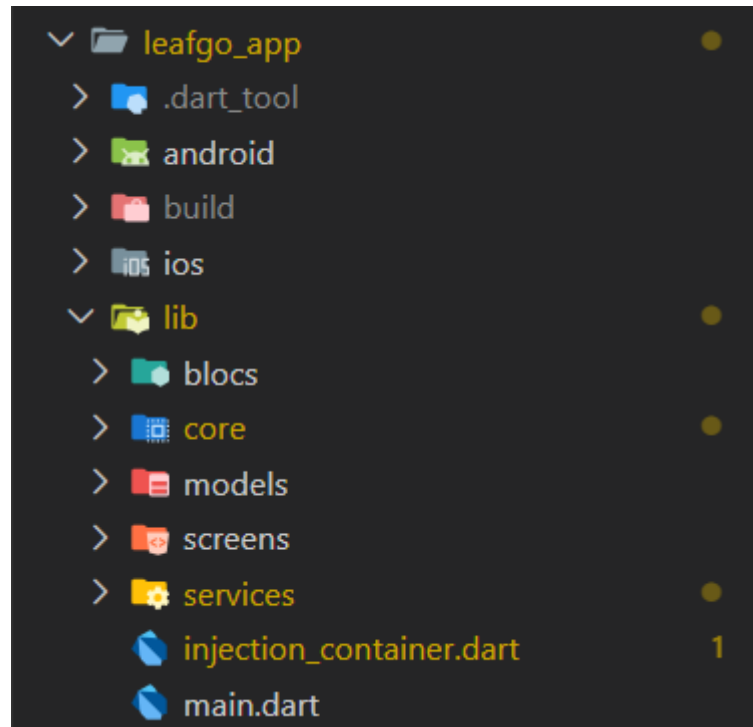


Hình 2.3 Mô hình Clean Architecture

Trong đề tài này, ứng dụng Flutter LeafGo được tổ chức theo Clean Architecture với ba tầng chính. Tầng Domain (lõi nghiệp vụ) chứa các Entities — mô hình dữ liệu thuần túy như TripModel, UserModel, DriverModel — và các Use Cases định nghĩa từng hành động nghiệp vụ cụ thể như đặt xe, hủy chuyến hay đánh giá tài xế. Tầng này hoàn toàn độc lập với Flutter và không phụ thuộc bất kỳ thư viện bên ngoài nào, giúp dễ dàng kiểm thử và tái sử dụng.

Tầng Data chịu trách nhiệm thu thập và cung cấp dữ liệu thông qua các Repository Implementation. Tại đây, dữ liệu từ REST API (.NET backend) được ánh xạ vào các Data Model, sau đó chuyển đổi sang Entity và trả về cho tầng Domain. Tầng Presentation quản lý giao diện và trạng thái ứng dụng, kết hợp widget Flutter

với cơ chế quản lý trạng thái phù hợp. Sự phân tách này giúp thay thế nguồn dữ liệu hay logic hiển thị mà không ảnh hưởng đến nghiệp vụ cốt lõi.



Hình 2.4 Cấu trúc thư mục Flutter project LeafGo theo Clean Architecture

2.4 ASP.NET Core Web API và SignalR

2.4.1 ASP.NET Core Web API

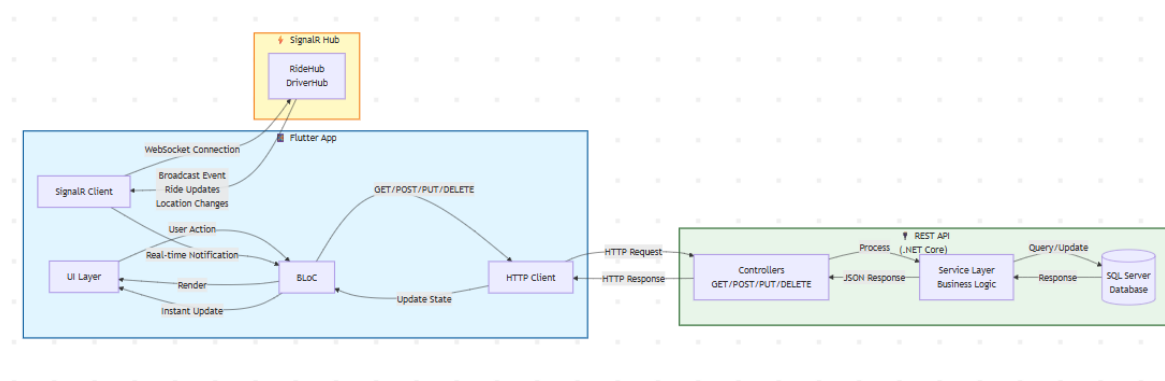
ASP.NET Core là framework phát triển web và API đa nền tảng, mã nguồn mở do Microsoft xây dựng và duy trì. ASP.NET Core Web API cho phép xây dựng các dịch vụ RESTful tuân theo kiến trúc client-server, trong đó server cung cấp các endpoint HTTP phản hồi theo định dạng JSON. Framework này có hiệu năng cao, hỗ trợ dependency injection tích hợp sẵn, middleware pipeline linh hoạt và khả năng triển khai đa môi trường. [5]

Trong hệ thống LeafGo, toàn bộ logic nghiệp vụ phía server - bao gồm xử lý yêu cầu đặt xe, quản lý tài khoản, tính cước và phân công tài xế - được triển khai dưới dạng các REST endpoint thông qua ASP.NET Core Web API. Ứng dụng Flutter giao tiếp với backend thông qua giao thức HTTP/HTTPS, sử dụng thư viện Dio trên Flutter để thực hiện các request GET, POST, PUT và DELETE.

2.4.2 SignalR

SignalR là thư viện của Microsoft tích hợp trong ASP.NET Core, cho phép xây dựng tính năng giao tiếp hai chiều theo thời gian thực giữa server và client. SignalR tự động lựa chọn phương thức truyền thông phù hợp nhất - WebSocket, Server-Sent Events hoặc Long Polling - tùy theo khả năng của môi trường mạng và trình duyệt, đảm bảo kết nối ổn định trong mọi điều kiện. [5]

Trong LeafGo, SignalR được sử dụng để đẩy thông báo trạng thái chuyển đi theo thời gian thực từ server xuống các client liên quan. Khi trạng thái một chuyến đi thay đổi - ví dụ tài xế nhận chuyến, tài xế đã đến nơi, hay chuyến đi hoàn thành - server phát sự kiện qua Hub SignalR, ứng dụng Flutter của khách hàng và tài xế nhận và cập nhật giao diện ngay lập tức mà không cần thực hiện polling định kỳ.



Hình 2.5 Luồng giao tiếp giữa Flutter client, REST API và SignalR Hub

2.5 Xác thực và phân quyền với JWT

JSON Web Token (JWT) là tiêu chuẩn mở (RFC 7519) định nghĩa cách truyền thông tin an toàn giữa các bên dưới dạng đối tượng JSON được ký số. Một JWT gồm ba phần: Header (thuật toán ký), Payload (dữ liệu claims như userId, role, thời hạn) và Signature (chữ ký xác thực tính toàn vẹn). JWT là stateless - server không cần lưu trữ session - phù hợp với kiến trúc REST API và ứng dụng di động. [7]

Trong hệ thống LeafGo, mỗi yêu cầu API từ Flutter app sau khi đăng nhập đều mang theo JWT trong header Authorization. Backend ASP.NET Core xác minh chữ ký của token và trích xuất thông tin phân quyền (customer, driver, admin) để kiểm

soát truy cập vào từng endpoint, đảm bảo mỗi nhóm người dùng chỉ thực hiện được các thao tác trong phạm vi quyền hạn của mình.

2.6 Cơ sở dữ liệu SQL Server

Microsoft SQL Server là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) do Microsoft phát triển, hỗ trợ ngôn ngữ truy vấn T-SQL, stored procedures, triggers, transaction và các cơ chế đảm bảo toàn vẹn dữ liệu theo chuẩn ACID. SQL Server cung cấp hiệu năng cao, công cụ quản trị trực quan (SQL Server Management Studio) và tích hợp tốt với hệ sinh thái .NET. [8]

Trong LeafGo, SQL Server lưu trữ toàn bộ dữ liệu nghiệp vụ của hệ thống bao gồm thông tin tài khoản người dùng và tài xế, dữ liệu phương tiện, lịch sử chuyến đi, cấu hình giá cước và lịch sử đánh giá. Ứng dụng ASP.NET Core giao tiếp với SQL Server thông qua Entity Framework Core - một ORM (Object-Relational Mapper) cho phép thao tác dữ liệu bằng cú pháp C# thay vì viết SQL trực tiếp, đồng thời hỗ trợ migrations để quản lý lịch sử thay đổi cấu trúc cơ sở dữ liệu.



Hình 2.6 SQL Server và Entity Framework Core trong hệ sinh thái .NET

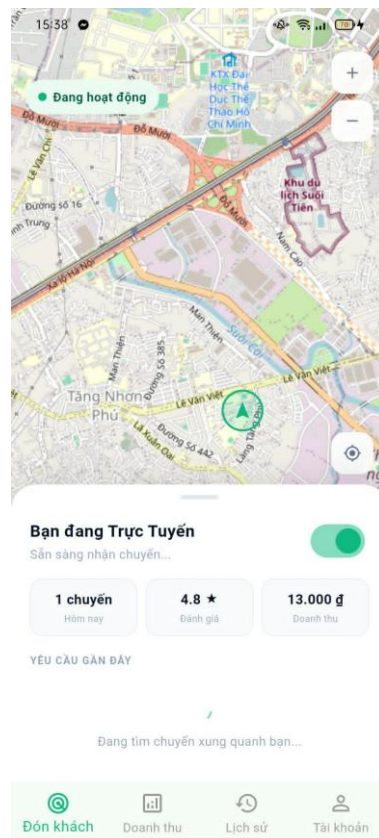
2.7 Nền tảng bản đồ: OpenStreetMap, Nominatim và OpenRouteService

2.7.1 OpenStreetMap và flutter_map

OpenStreetMap (OSM) là dự án bản đồ cộng đồng mã nguồn mở, cung cấp dữ liệu địa lý miễn phí và liên tục được cập nhật bởi hàng triệu tình nguyện viên trên

toàn thế giới. Dữ liệu OSM có thể sử dụng tự do theo giấy phép ODbL, không phát sinh chi phí theo lượng truy vấn như các dịch vụ bản đồ thương mại. [9]

Trong ứng dụng Flutter, bản đồ được hiển thị thông qua thư viện *flutter_map* (phiên bản 8.0.1) - một widget Flutter render tile map từ các tile server tương thích OSM. Thư viện *latlong2* (phiên bản 0.9.1) được dùng để biểu diễn và tính toán tọa độ địa lý, còn *geolocator* (phiên bản 13.0.2) xử lý việc lấy vị trí GPS hiện tại của thiết bị. Bộ ba thư viện này kết hợp cung cấp đầy đủ nền tảng bản đồ cho LeafGo mà không phát sinh chi phí API.



Hình 2.7 Hiển thị bản đồ OpenStreetMap qua *flutter_map*

2.7.2 Nominatim API

Nominatim là dịch vụ geocoding và tìm kiếm địa điểm miễn phí được xây dựng trên nền dữ liệu OpenStreetMap, do OpenStreetMap Foundation vận hành. Nominatim cung cấp hai chức năng chính: forward geocoding - chuyển đổi chuỗi địa

chỉ văn bản thành tọa độ latitude/longitude, và reverse geocoding - chuyển tọa độ thành địa chỉ có thể đọc được. [9]

Trong LeafGo, Nominatim API được gọi từ LocationService mỗi khi người dùng nhập địa chỉ điểm đón hoặc điểm đến. Kết quả tìm kiếm được giới hạn trong lãnh thổ Việt Nam thông qua tham số countrycodes=vn để nâng cao độ chính xác. Tọa độ trả về sau đó được dùng để đặt marker trên bản đồ và gửi lên server khi tạo chuyến đi.

2.7.3 OpenRouteService

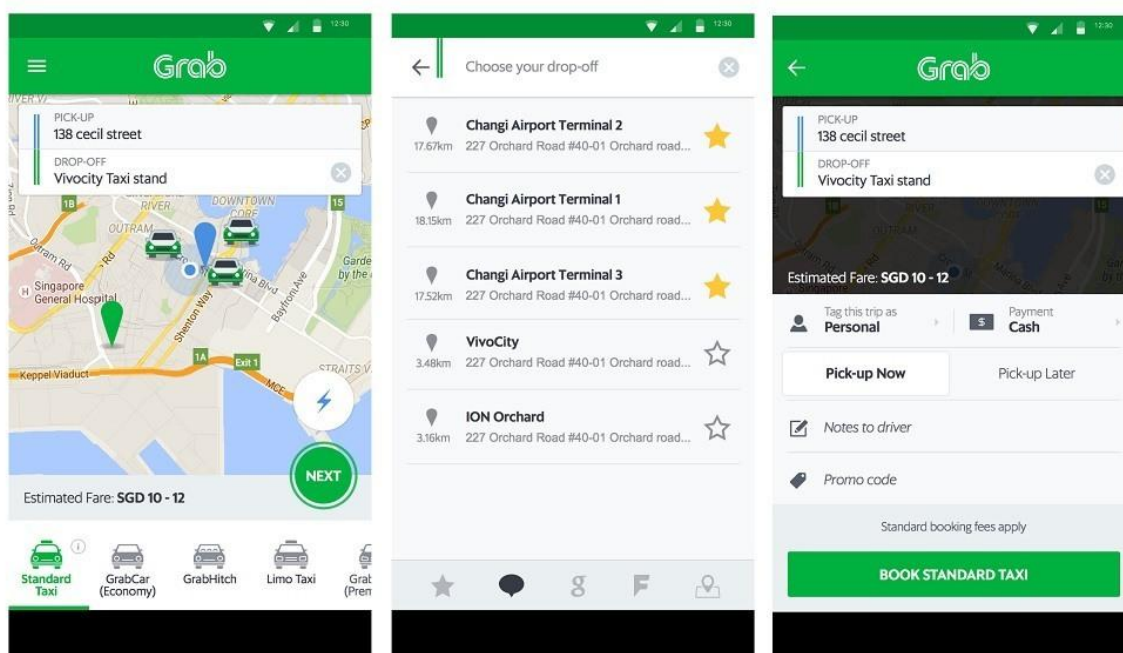
OpenRouteService (ORS) là dịch vụ định tuyến địa lý (routing) mã nguồn mở do HeiGIT Institute (Đại học Heidelberg, Đức) phát triển và vận hành, sử dụng dữ liệu OpenStreetMap. ORS cung cấp API tính toán tuyến đường tối ưu giữa hai hoặc nhiều điểm, hỗ trợ nhiều phương thức di chuyển (ô tô, xe máy, xe đạp, đi bộ) và trả về kết quả dưới định dạng GeoJSON. [10]

LeafGo sử dụng OpenRouteService Directions API để tính toán và vẽ lộ trình từ điểm đón đến điểm đến trên bản đồ. Kết quả tuyến đường dạng polyline được vẽ đè lên lớp tile map của flutter_map, cho phép khách hàng và tài xế quan sát trực quan lộ trình di chuyển cùng với khoảng cách và thời gian ước tính.

2.8 Khảo sát các ứng dụng đặt xe tiêu biểu

2.8.1 Ứng dụng Grab

Grab là nền tảng đặt xe công nghệ hàng đầu Đông Nam Á, được thành lập năm 2012 tại Malaysia, hiện hoạt động tại 8 quốc gia với hơn 35 triệu người dùng hàng tháng. Tại Việt Nam, Grab chiếm thị phần lớn nhất trong phân khúc gọi xe và giao đồ ăn. Ứng dụng Grab cung cấp trải nghiệm đặt xe mượt mà với tính năng định vị thời gian thực, ước tính giá trước chuyến, nhiều phương thức thanh toán và hệ thống đánh giá tài xế minh bạch. [11]



Hình 2.8 Giao diện đặt xe trên ứng dụng Grab

Điểm mạnh của Grab là hệ sinh thái dịch vụ đa dạng (xe, đồ ăn, thanh toán), thuật toán ghép tài xế tinh vi và mạng lưới tài xế rộng lớn. Tuy nhiên, hoa hồng thu từ tài xế khá cao (20-25%), ứng dụng có nhiều tính năng dẫn đến kích thước lớn và tốc độ khởi động chậm trên thiết bị cấu hình thấp. Đối với các đơn vị vận tải nhỏ muốn xây dựng nền tảng riêng, chi phí gia nhập và phụ thuộc vào hạ tầng của Grab là rào cản lớn.

2.8.2 Ứng dụng Be

Be (Be Group) là ứng dụng gọi xe thuần Việt, ra mắt năm 2018, định vị là lựa chọn thay thế nội địa cho Grab với chính sách hoa hồng thấp hơn và tập trung vào thị trường Việt Nam. Be hỗ trợ các dịch vụ beBike, beCar và beDelivery, tích hợp ví điện tử và cho phép tài xế nhận thanh toán tiền mặt hoặc chuyển khoản. [12]

Ưu điểm của Be là giao diện gọn nhẹ, thân thiện với tài xế và phù hợp với thị trường nội địa. Hạn chế là mạng lưới tài xế nhỏ hơn Grab ở nhiều khu vực, tính năng theo dõi chuyến đi thời gian thực chưa mượt bằng và chưa có hệ sinh thái dịch vụ đa dạng.



Hình 2.9 Giao diện đặt xe trên ứng dụng Be

2.8.3 Nhận xét và định hướng đề tài

Qua khảo sát hai ứng dụng tiêu biểu trên, có thể rút ra một số nhận xét chung. Các ứng dụng thương mại hiện tại đều được xây dựng trên hạ tầng phức tạp, quy mô lớn và không phù hợp để nghiên cứu hay triển khai ở quy mô nhỏ. Cả Grab và Be đều sử dụng Google Maps Platform với chi phí cao; việc tích hợp các dịch vụ bản đồ mã nguồn mở như OpenStreetMap và OpenRouteService là hướng đi kinh tế và linh hoạt hơn cho các đơn vị vừa và nhỏ.

Đề tài LeafGo tiếp cận bài toán theo hướng xây dựng một hệ thống đặt xe đầy đủ chức năng cốt lõi nhưng sử dụng hoàn toàn công nghệ mã nguồn mở và miễn phí cho phần bản đồ (OpenStreetMap, Nominatim, OpenRouteService), đồng thời áp dụng kiến trúc Clean Architecture và bộ công nghệ .NET - Flutter hiện đại để đảm bảo khả năng bảo trì và mở rộng. Đây là điểm khác biệt và đóng góp chính so với các công bố liên quan.

2.9 Kết luận và lựa chọn công nghệ cho đề tài

Từ khảo sát lý thuyết và phân tích thực tiễn ở các mục trên, nhóm xác định bộ công nghệ chính thức cho hệ thống LeafGo như sau. Phía ứng dụng di động: Flutter (Dart) với kiến trúc Clean Architecture, thư viện flutter_map cho bản đồ, geolocator cho GPS, Dio cho gọi API và signalr_client cho kết nối realtime. Phía backend:

ASP.NET Core Web API với SignalR Hub, xác thực kép Firebase Authentication và JWT. Lưu trữ dữ liệu: SQL Server thông qua Entity Framework Core. Dịch vụ bản đồ: Nominatim API cho geocoding, OpenRouteService cho tính toán lộ trình, tile server OpenStreetMap cho hiển thị bản đồ.

Sự lựa chọn này đảm bảo cân bằng giữa ba tiêu chí: hiệu năng và trải nghiệm người dùng (Flutter, ASP.NET Core), tính bảo mật và tin cậy (Firebase, JWT, SQL Server), và chi phí triển khai thấp (toàn bộ dịch vụ bản đồ miễn phí). Đây là nền tảng vững chắc để triển khai hệ thống LeafGo đầy đủ được trình bày trong các chương tiếp theo.

Kết luận: Chương 2 đã trình bày toàn bộ nền tảng lý thuyết và công nghệ của đề tài, bao gồm Flutter/Dart với Clean Architecture, ASP.NET Core + SignalR, xác thực JWT + Firebase, SQL Server và hệ thống bản đồ mã nguồn mở. Qua khảo sát Grab và Be, nhóm xác định được hướng phát triển phù hợp và những đóng góp cụ thể của LeafGo. Đây là cơ sở lý thuyết trực tiếp phục vụ cho quá trình phân tích, thiết kế và triển khai hệ thống được trình bày trong Chương 3 và Chương 4.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

Chương này trình bày toàn bộ quá trình phân tích và thiết kế hệ thống LeafGo. Nội dung bao gồm mô tả bài toán và xác định yêu cầu hệ thống, phân tích các tác nhân tham gia, thiết kế các sơ đồ UML mô tả luồng nghiệp vụ và tương tác, thiết kế cơ sở dữ liệu, kiến trúc hệ thống và giao diện người dùng. Các kết quả của chương này là cơ sở trực tiếp để triển khai hệ thống trong Chương 4.

3.1 Mô tả bài toán

Hệ thống đặt xe công nghệ truyền thống gặp nhiều hạn chế trong việc kết nối khách hàng với tài xế một cách nhanh chóng, minh bạch và đáng tin cậy. Khách hàng khó biết trước chi phí, không theo dõi được vị trí tài xế theo thời gian thực và thiếu cơ chế phản hồi sau chuyến đi. Về phía tài xế, việc nhận chuyến thủ công gây lãng phí thời gian và không có công cụ thống kê thu nhập hiệu quả. Đơn vị vận hành cũng thiếu hệ thống giám sát tập trung để quản lý hoạt động toàn hệ thống.

Bài toán đặt ra là xây dựng một ứng dụng di động trên nền tảng Android cho phép ba nhóm người dùng - khách hàng, tài xế và quản trị viên - tương tác trong một hệ thống thống nhất. Khách hàng có thể đặt xe, theo dõi chuyến đi theo thời gian thực và đánh giá dịch vụ. Tài xế nhận thông báo chuyến đi tức thì, cập nhật trạng thái và xem thống kê thu nhập. Quản trị viên quản lý toàn bộ tài khoản, điều chỉnh giá cước và giám sát hoạt động hệ thống.

Hệ thống LeafGo được xây dựng theo mô hình client-server, trong đó ứng dụng Flutter Android đóng vai trò client giao tiếp với backend ASP.NET Core qua REST API và SignalR. Input của bài toán là thông tin vị trí, yêu cầu đặt xe và dữ liệu người dùng; output là trạng thái chuyến đi, lộ trình bản đồ và kết quả thống kê. Phạm vi triển khai giới hạn trên Android, hỗ trợ từ Android 8.0 trở lên và chỉ phục vụ khu vực địa lý tại Việt Nam.

3.2 Phân tích yêu cầu hệ thống

3.2.1 Yêu cầu chức năng

Yêu cầu chức năng của hệ thống LeafGo được phân chia theo ba nhóm người dùng chính. Mỗi nhóm có tập chức năng riêng biệt, phù hợp với vai trò và quyền hạn trong hệ thống.

Nhóm chức năng dành cho Khách hàng: Khách hàng có thể đăng ký tài khoản bằng email và mật khẩu thông qua Firebase Authentication, đăng nhập và đăng xuất khỏi ứng dụng. Sau khi đăng nhập, khách hàng nhập địa chỉ điểm đón và điểm đến để hệ thống tính toán lộ trình và hiển thị chi phí ước tính trước khi xác nhận đặt xe. Trong quá trình chờ và di chuyển, khách hàng theo dõi vị trí tài xế theo thời gian thực trên bản đồ. Sau khi chuyến đi hoàn thành, khách hàng có thể đánh giá tài xế từ 1 đến 5 sao kèm nhận xét, đồng thời xem lại toàn bộ lịch sử chuyến đi đã thực hiện.

Nhóm chức năng dành cho Tài xế: Tài xế đăng ký tài khoản và cập nhật thông tin phương tiện (loại xe, biển số, hãng xe, màu sắc) trước khi bắt đầu nhận chuyến. Khi ở trạng thái trực tuyến, tài xế nhận thông báo realtime về các yêu cầu đặt xe mới và có thể chấp nhận hoặc từ chối từng chuyến. Trong quá trình thực hiện chuyến, tài xế cập nhật tuần tự các trạng thái: đã đến điểm đón, đang di chuyển, và hoàn thành chuyến. Ngoài ra, tài xế xem thống kê thu nhập theo ngày và tháng, cùng với lịch sử toàn bộ chuyến đi hoàn thành.

Nhóm chức năng dành cho Quản trị viên: Quản trị viên đăng nhập vào hệ thống với tài khoản có quyền admin. Từ bảng điều khiển, quản trị viên quản lý toàn bộ tài khoản khách hàng và tài xế (xem, khóa, mở khóa tài khoản), điều chỉnh cấu hình giá cước, theo dõi tất cả các chuyến đi đang diễn ra và đã hoàn thành, đồng thời xem thống kê tổng quan về hoạt động của hệ thống.

Bảng 3.1 Danh sách yêu cầu chức năng hệ thống LeafGo

Mã	Chức năng	Mô tả	Actor
UC01	Đăng ký / Đăng nhập	Xác thực tài khoản qua Firebase Auth	KH, TX
UC02	Đặt xe	Nhập điểm đón/đến, xem lộ trình và giá, xác nhận	KH

UC03	Theo dõi chuyển đi	Xem vị trí taxi realtime trên bản đồ	KH
UC04	Hủy chuyển	Hủy chuyển trước khi taxi đến	KH
UC05	Đánh giá taxi	Chấm sao và gửi nhận xét sau chuyến	KH
UC06	Xem lịch sử chuyến	Tra cứu danh sách chuyến đã đi/đã thực hiện	KH, TX
UC07	Nhận/Từ chối chuyến	Xem thông tin chuyến và quyết định nhận/từ chối	TX
UC08	Cập nhật trạng thái chuyến	Chuyển trạng thái: Đã đến → Đang đi → Hoàn thành	TX
UC09	Cập nhật thông tin xe	Nhập biển số, loại xe, màu sắc trước khi nhận chuyến	TX
UC10	Xem thống kê thu nhập	Thống kê doanh thu theo ngày, tháng	TX
UC11	Quản lý tài khoản	Xem, khóa, mở khóa tài khoản KH và TX	Admin
UC12	Cấu hình giá cước	Điều chỉnh đơn giá và các tham số tính tiền	Admin
UC13	Giám sát hệ thống	Xem thống kê chuyến đi và hoạt động toàn hệ thống	Admin

3.2.2 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống LeafGo cần đáp ứng một số tiêu chí phi chức năng quan trọng. Về hiệu năng, thời gian phản hồi của API không vượt quá 2 giây trong điều kiện mạng 4G bình thường; thông báo trạng thái chuyến đi qua SignalR cần được đẩy đến client trong vòng dưới 1 giây. Về tính ổn định, ứng dụng cần hoạt động liên tục mà không bị crash trong các luồng sử dụng chính; dữ liệu chuyến đi phải được lưu trữ nhất quán ngay cả khi có sự cố kết nối tạm thời.

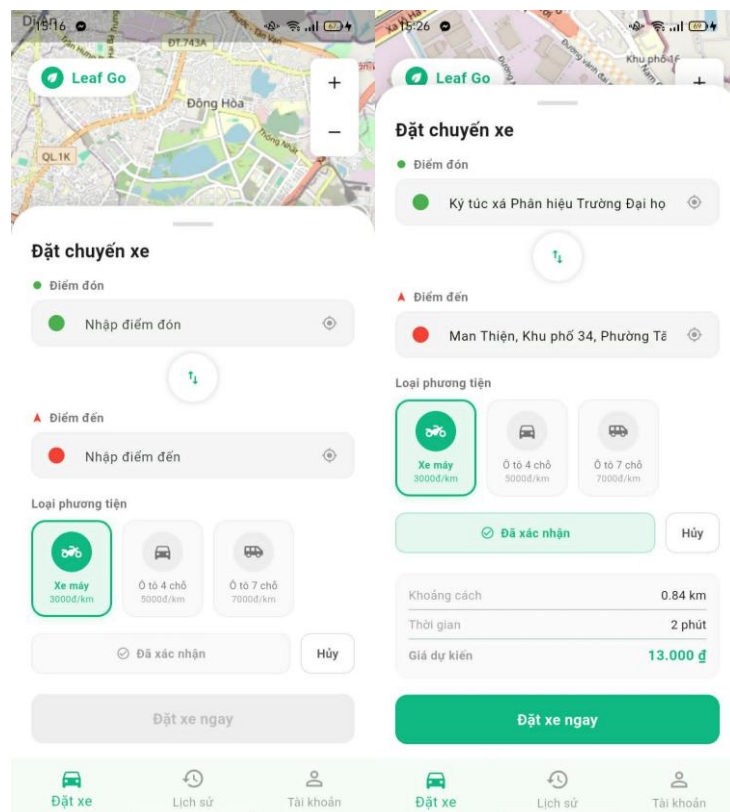
Về bảo mật, toàn bộ giao tiếp giữa client và server được mã hóa qua HTTPS; xác thực hai lớp bằng Firebase Authentication và JWT đảm bảo chỉ người dùng hợp lệ mới truy cập được hệ thống; mỗi vai trò chỉ được phép truy cập các endpoint thuộc phạm vi quyền hạn của mình. Về khả năng sử dụng, giao diện được thiết kế theo nguyên tắc Material Design, thân thiện với người dùng phổ thông và hoạt động tốt trên các thiết bị Android phổ biến với màn hình từ 5 inch trở lên, độ phân giải tối thiểu 720×1280 pixel.

3.3 Thiết kế giao diện người dùng

Giao diện ứng dụng LeafGo được thiết kế theo nguyên tắc Material Design 3 của Google, đảm bảo tính nhất quán, trực quan và thân thiện với người dùng Android. Toàn bộ giao diện được chia thành ba luồng độc lập tương ứng với ba vai trò: luồng khách hàng, luồng tài xế và luồng quản trị viên. Sau khi đăng nhập, ứng dụng tự động điều hướng người dùng đến giao diện phù hợp với vai trò được gán cho tài khoản.

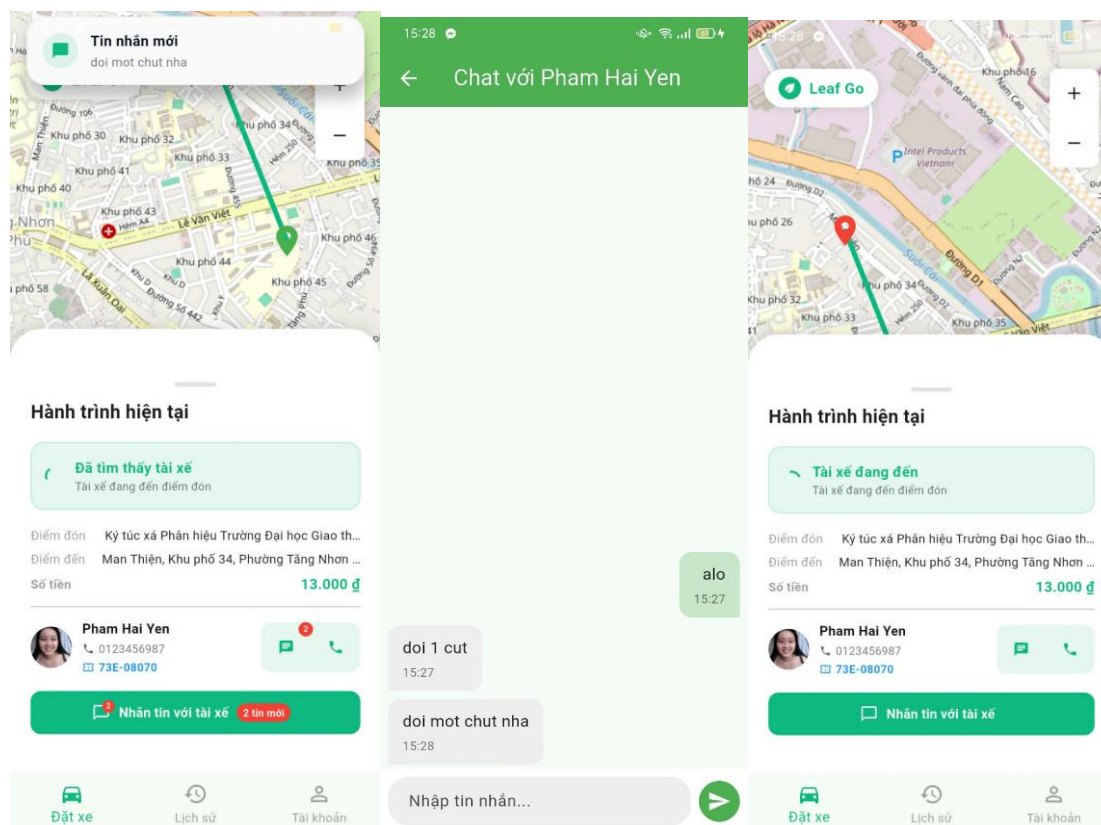
3.3.1 Giao diện Khách hàng

Màn hình chính của khách hàng hiển thị bản đồ OpenStreetMap toàn màn hình với hai ô nhập liệu ở phía trên cho phép tìm kiếm và chọn địa chỉ điểm đón và điểm đến thông qua Nominatim API. Sau khi hai điểm được xác định, hệ thống vẽ lộ trình OpenRouteService lên bản đồ và hiển thị thông tin ước tính khoảng cách, thời gian di chuyển và chi phí chuyến đi. Khách hàng nhấn xác nhận để gửi yêu cầu và chuyển sang màn hình theo dõi tài xế.



Hình 3.1 Màn hình đặt xe của khách hàng trong ứng dụng LeafGo

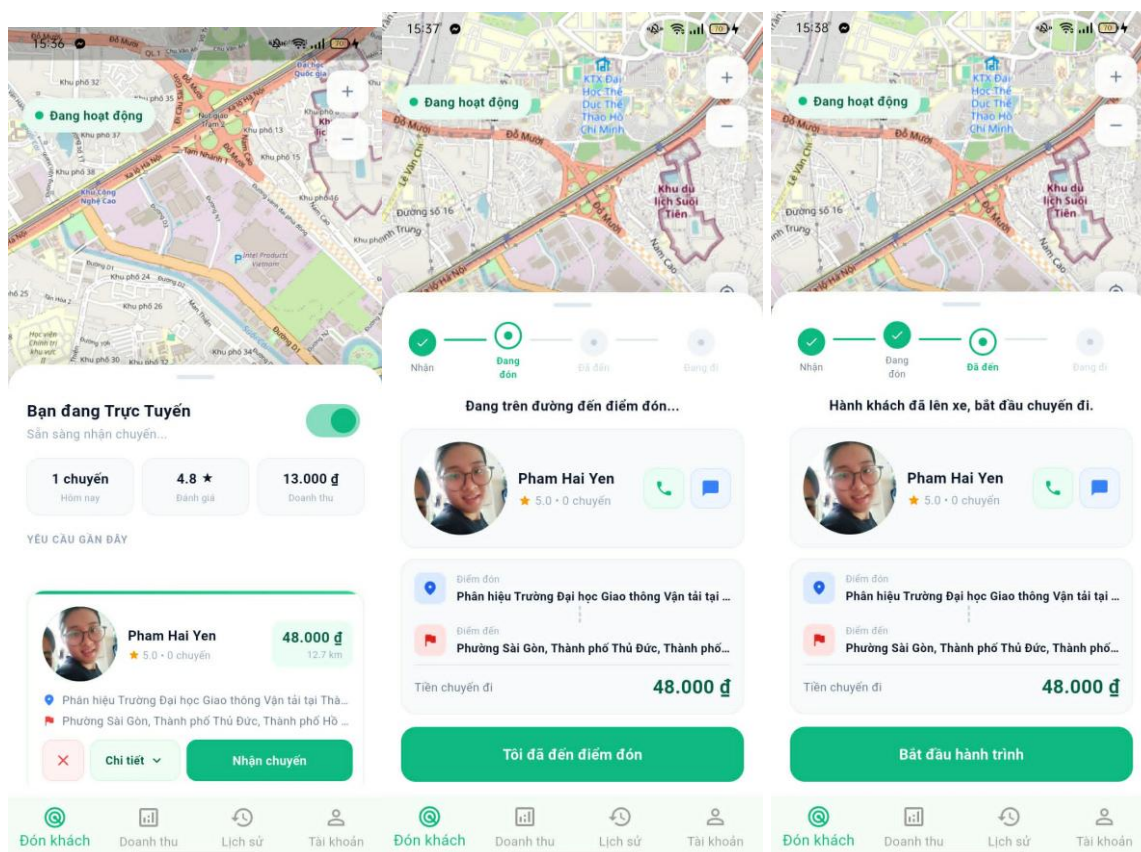
Màn hình theo dõi chuyến đi hiển thị vị trí tài xế di chuyển trên bản đồ theo thời gian thực, cùng với thông tin tài xế (tên, số điện thoại, biển số xe, điểm đánh giá) và trạng thái hiện tại của chuyến. Sau khi chuyến hoàn thành, màn hình đánh giá xuất hiện cho phép khách hàng chấm sao và để lại nhận xét trước khi quay về màn hình chính.



Hình 3.2 Màn hình theo dõi chuyến đi và đánh giá tài xế sau chuyến

3.3.2 Giao diện Tài xế

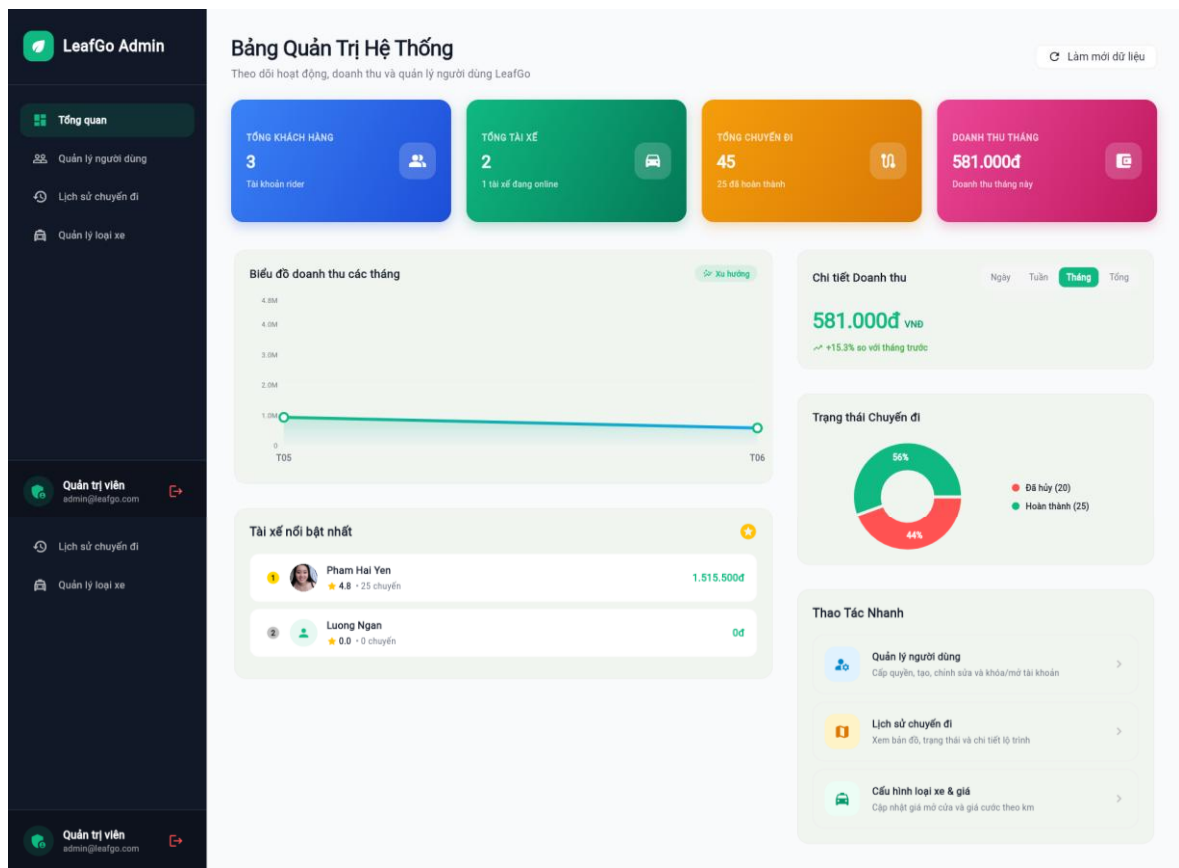
Màn hình chính của tài xế hiển thị nút bật/tắt trạng thái trực tuyến cùng với bản đồ thể hiện vị trí hiện tại. Khi có chuyến đi mới, một hộp thoại xuất hiện hiển thị đầy đủ thông tin gồm tên và số điện thoại khách hàng, địa chỉ điểm đón và điểm đến, khoảng cách và thu nhập dự kiến. Tài xế có thể nhấn “Nhận chuyến” hoặc “Từ chối” ngay trên hộp thoại này.



Hình 3.3 Giao diện nhận chuyến và cập nhật trạng thái chuyến đi của tài xế

3.3.3 Giao diện Quản trị viên

Giao diện quản trị viên được thiết kế dạng dashboard với menu điều hướng bên trái gồm các mục: Tổng quan, Quản lý tài xế, Quản lý khách hàng, Cấu hình giá và Thống kê. Màn hình tổng quan hiển thị các chỉ số chính của hệ thống như số chuyến đi trong ngày, doanh thu, số tài xế đang hoạt động và số khách hàng mới. Các màn hình quản lý hiển thị danh sách dạng bảng kèm chức năng tìm kiếm, lọc và thao tác khóa/mở khóa tài khoản.

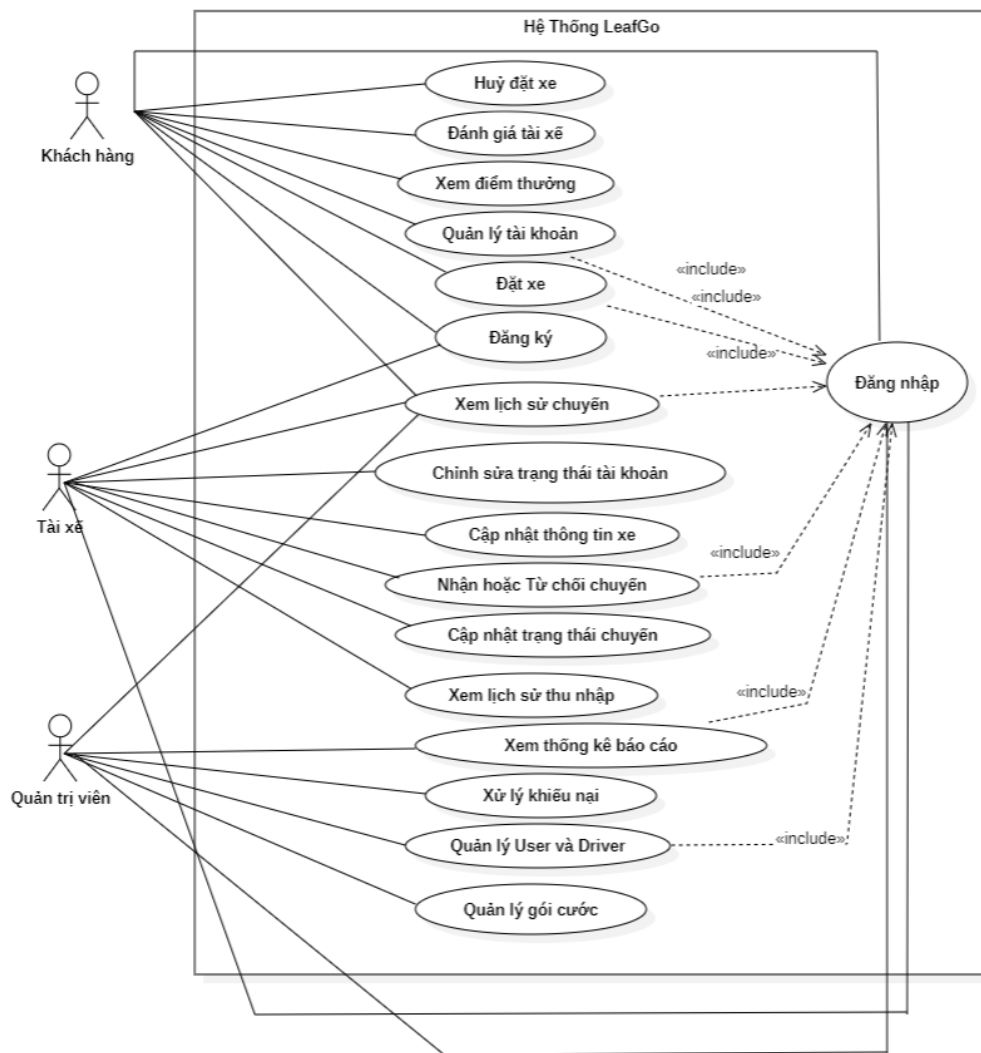


Hình 3.4 Giao diện bảng điều khiển của Quản trị viên trong ứng dụng LeafGo

3.4 Phân tích và thiết kế bằng UML

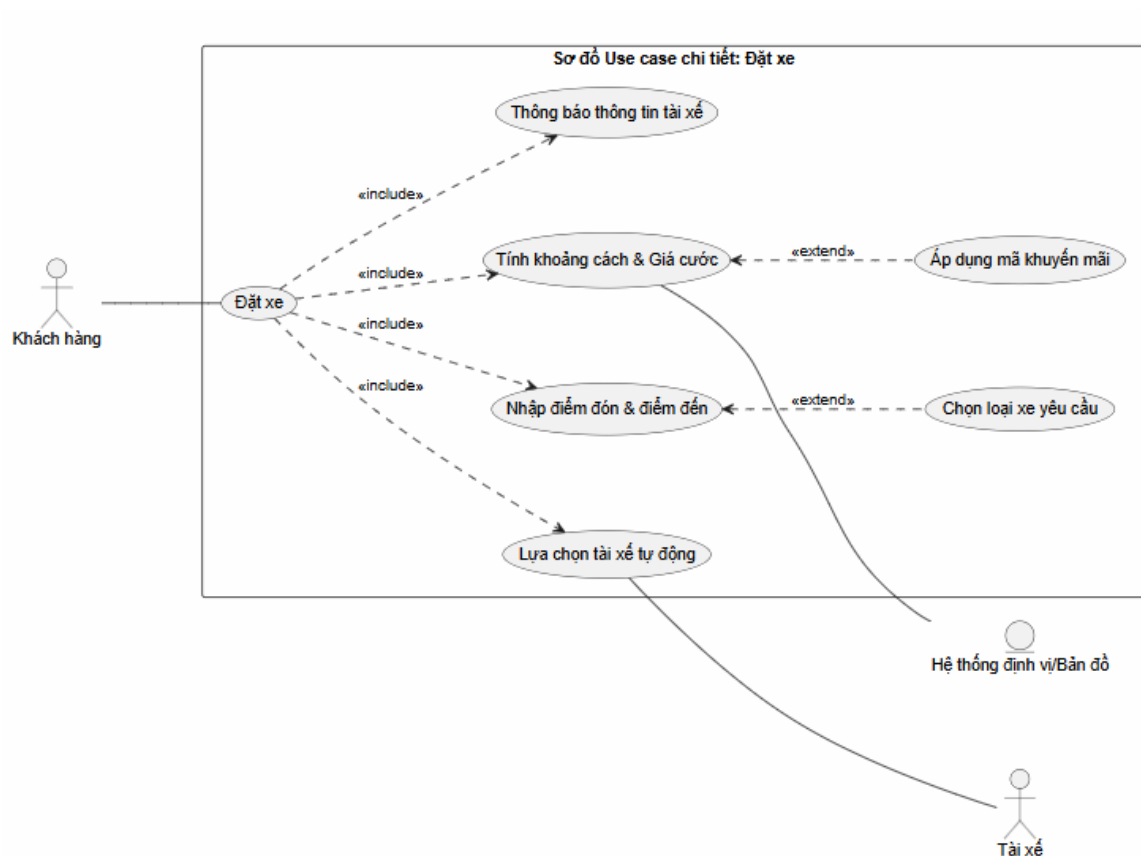
3.4.1 Sơ đồ Use Case

Sơ đồ Use Case tổng quát mô tả toàn bộ các chức năng của hệ thống và mối quan hệ giữa các actor với từng use case. Ba actor chính là Khách hàng, Tài xế và Quản trị viên, mỗi actor tương tác với tập hợp các use case thuộc phạm vi quyền hạn của mình như đã trình bày trong Bảng 3.1. Các use case liên quan đến xác thực (đăng ký, đăng nhập) được dùng chung bởi cả Khách hàng và Tài xế thông qua quan hệ include.

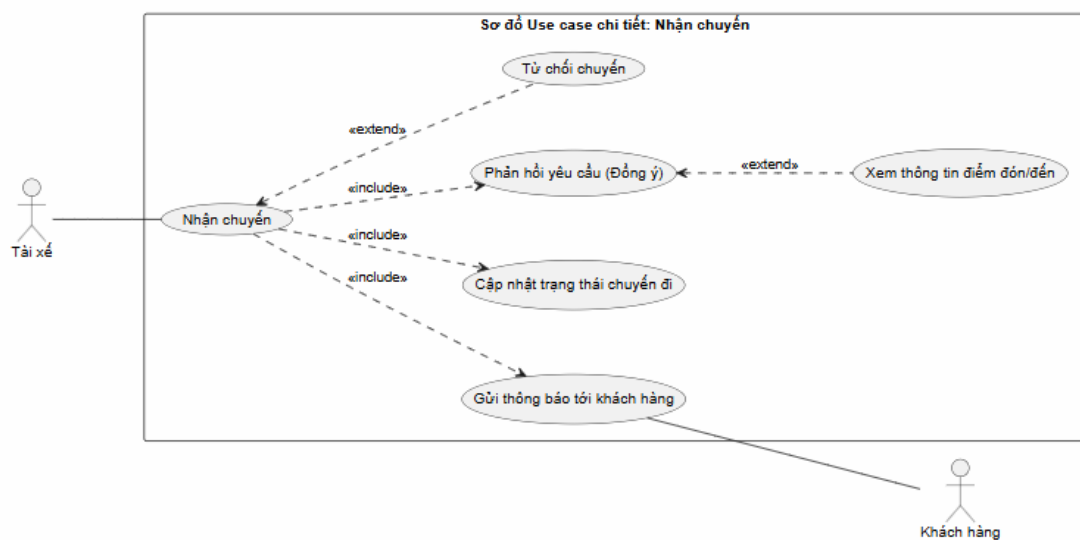


Hình 3.5 Sơ đồ Use Case tổng quát của hệ thống LeafGo

Trong các use case, nhóm ưu tiên mô tả chi tiết hai use case quan trọng nhất là Đặt xe (UC02) và Nhận/Từ chối chuyến (UC07) vì đây là luồng nghiệp vụ cốt lõi quyết định trải nghiệm người dùng. Use case Đặt xe bao gồm các bước: khách hàng nhập điểm đón/đến → hệ thống tìm lộ trình → hiển thị giá → xác nhận → tìm tài xế → thông báo realtime. Use case Nhận chuyến bao gồm: nhận thông báo → xem thông tin → chấp nhận/từ chối → cập nhật trạng thái.



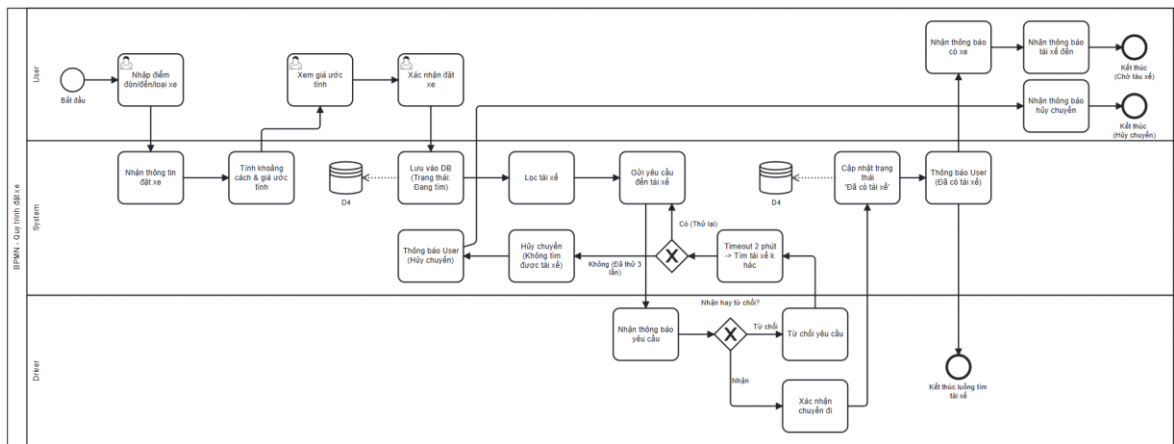
Hình 3.6 Sơ đồ Use Case chi tiết cho UC02 (Đặt xe)



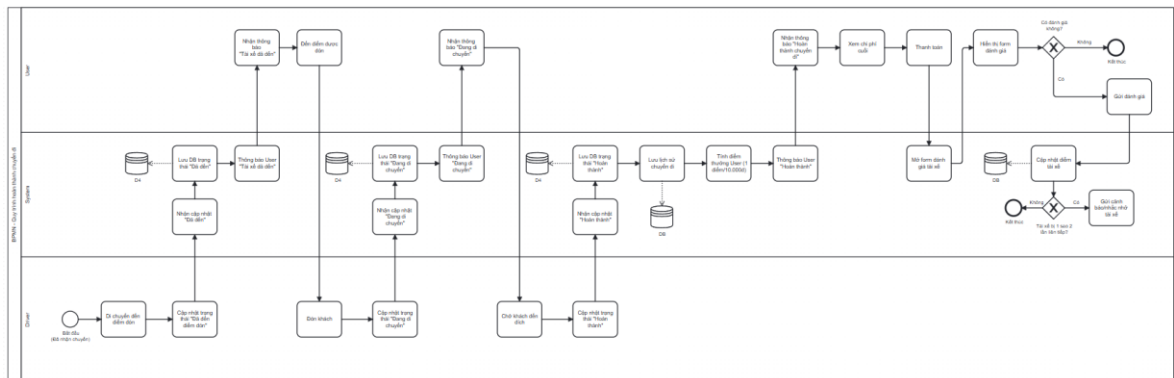
Hình 3.7 Sơ đồ Use Case chi tiết cho UC07 (Nhận chuyển)

3.4.2 Sơ đồ BPMN (Business Process Model and Notation)

Sơ đồ BPMN mô tả luồng xử lý nghiệp vụ của quy trình đặt xe từ góc độ toàn hệ thống. Luồng bắt đầu khi khách hàng nhập thông tin chuyến đi và kết thúc khi cả hai bên (khách hàng và tài xế) nhận được xác nhận hoàn thành. Sơ đồ thể hiện rõ các nhánh rẽ quan trọng: trường hợp không tìm được tài xế trong thời gian quy định (timeout), trường hợp tài xế từ chối chuyển dẫn đến việc hệ thống tìm tài xế khác, và trường hợp khách hàng hủy chuyến khi đang trong trạng thái chờ.



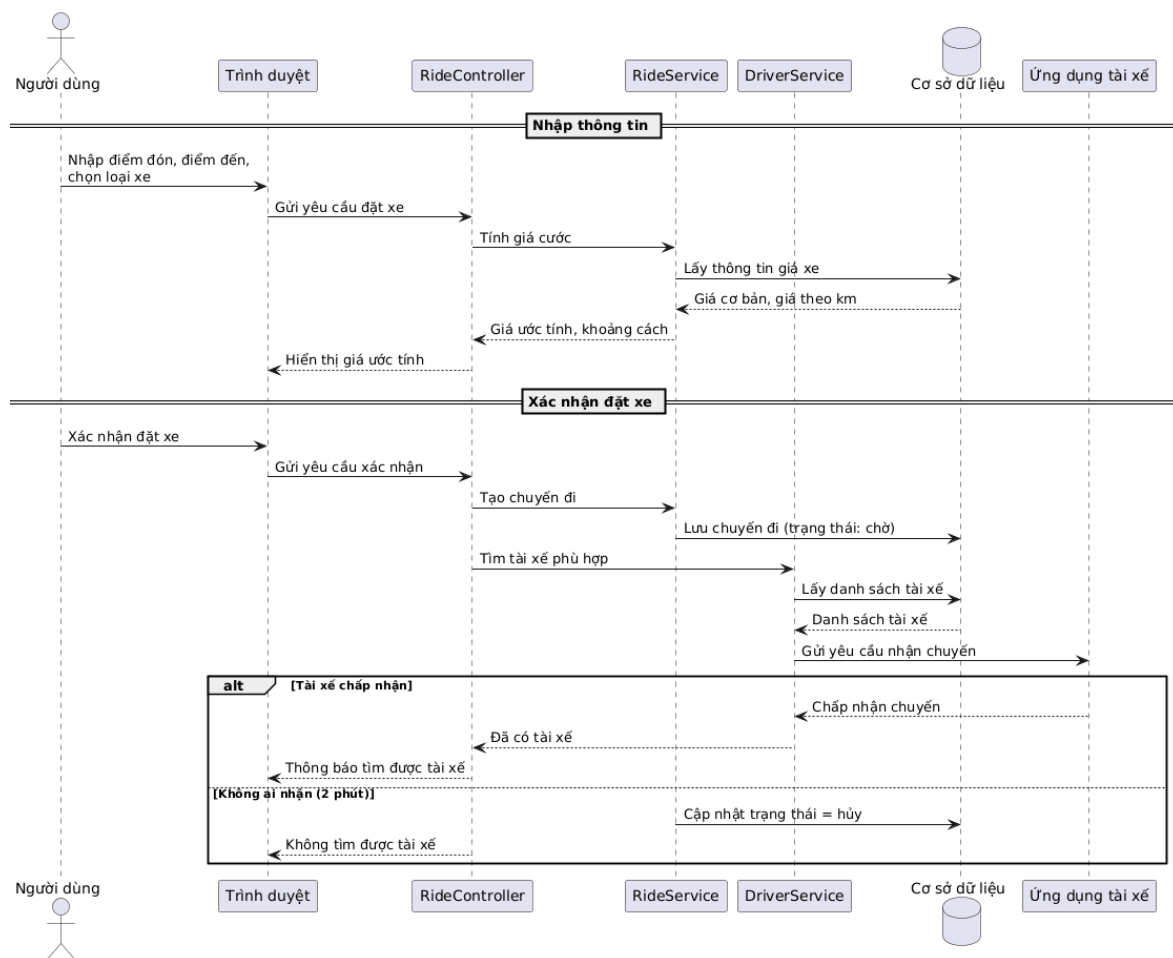
Hình 3.8 Sơ đồ BPMN mô tả luồng xử lý quy trình đặt xe



Hình 3.9 Sơ đồ BPMN mô tả luồng xử lý quy trình hoàn thành chuyến

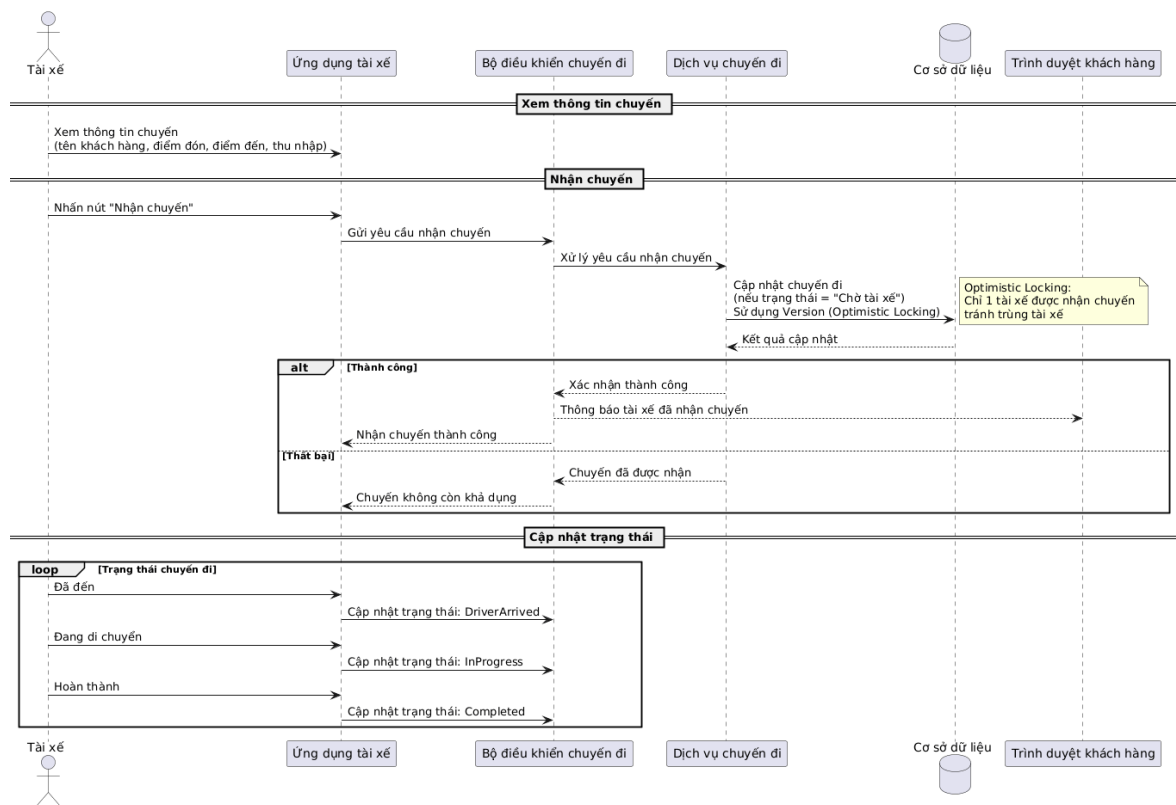
3.4.3 Sơ đồ Sequence (Tuần tự)

Sơ đồ Sequence mô tả chi tiết tương tác theo thứ tự thời gian giữa các thành phần hệ thống cho từng luồng chức năng quan trọng. Nhóm tập trung vẽ sơ đồ Sequence cho ba luồng tiêu biểu: đăng nhập (thể hiện tương tác giữa Flutter App, Firebase Auth và .NET API), đặt xe (thể hiện toàn bộ chuỗi gọi API và sự kiện SignalR), và cập nhật trạng thái chuyến đi (thể hiện cơ chế realtime notification).



Hình 3.10 Sơ đồ Sequence mô tả luồng đặt xe từ

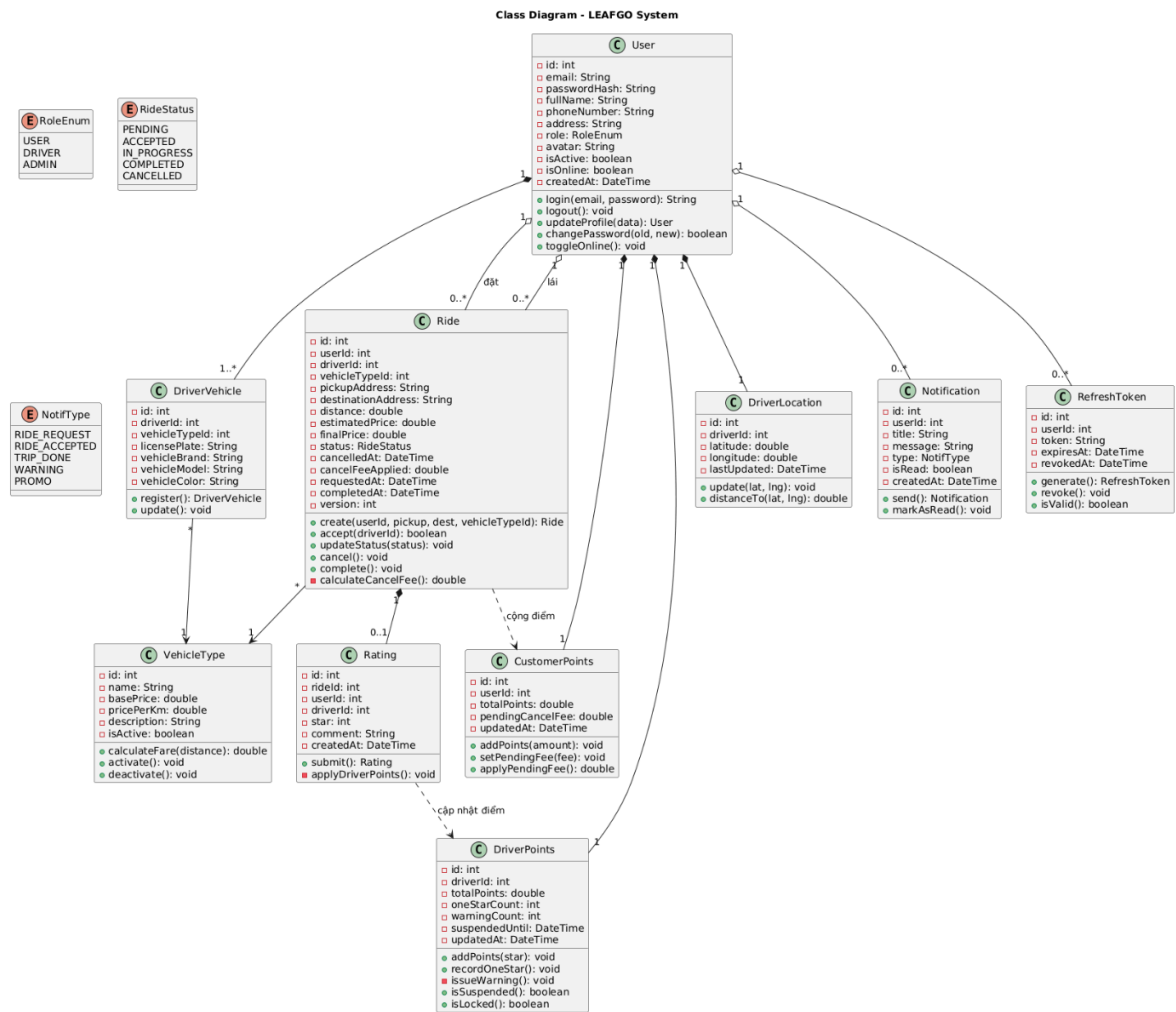
Trong sơ đồ Sequence luồng đặt xe, Flutter App gửi request tạo chuyến đi đến REST API kèm JWT token. API xác thực token, tạo bản ghi chuyến đi trong SQL Server với trạng thái Pending, sau đó phát sự kiện qua SignalR Hub đến tất cả tài xế đang trực tuyến trong khu vực. Khi tài xế nhấn nhận chuyến, ứng dụng tài xế gửi request cập nhật trạng thái, API cập nhật SQL Server và phát tiếp sự kiện SignalR đến ứng dụng khách hàng để cập nhật giao diện theo thời gian thực.



Hình 3.11 Sơ đồ Sequence khi tài xế cập nhật trạng thái

3.4.4 Sơ đồ Class

Sơ đồ Class mô tả cấu trúc các lớp nghiệp vụ chính và mối quan hệ giữa chúng trong hệ thống LeafGo. Các lớp Entity trung tâm gồm User (với các thuộc tính cơ bản: id, email, phone, role, isActive), Driver (kế thừa từ User, bổ sung thêm: vehicleInfo, isOnline, rating), Trip (id, customerId, driverId, pickupLocation, dropoffLocation, status, fare, createdAt) và Rating (id, tripId, score, comment). Quan hệ giữa các lớp: một User (khách hàng) có thể tạo nhiều Trip; mỗi Trip được phân công cho một Driver; mỗi Trip có tối đa một Rating.

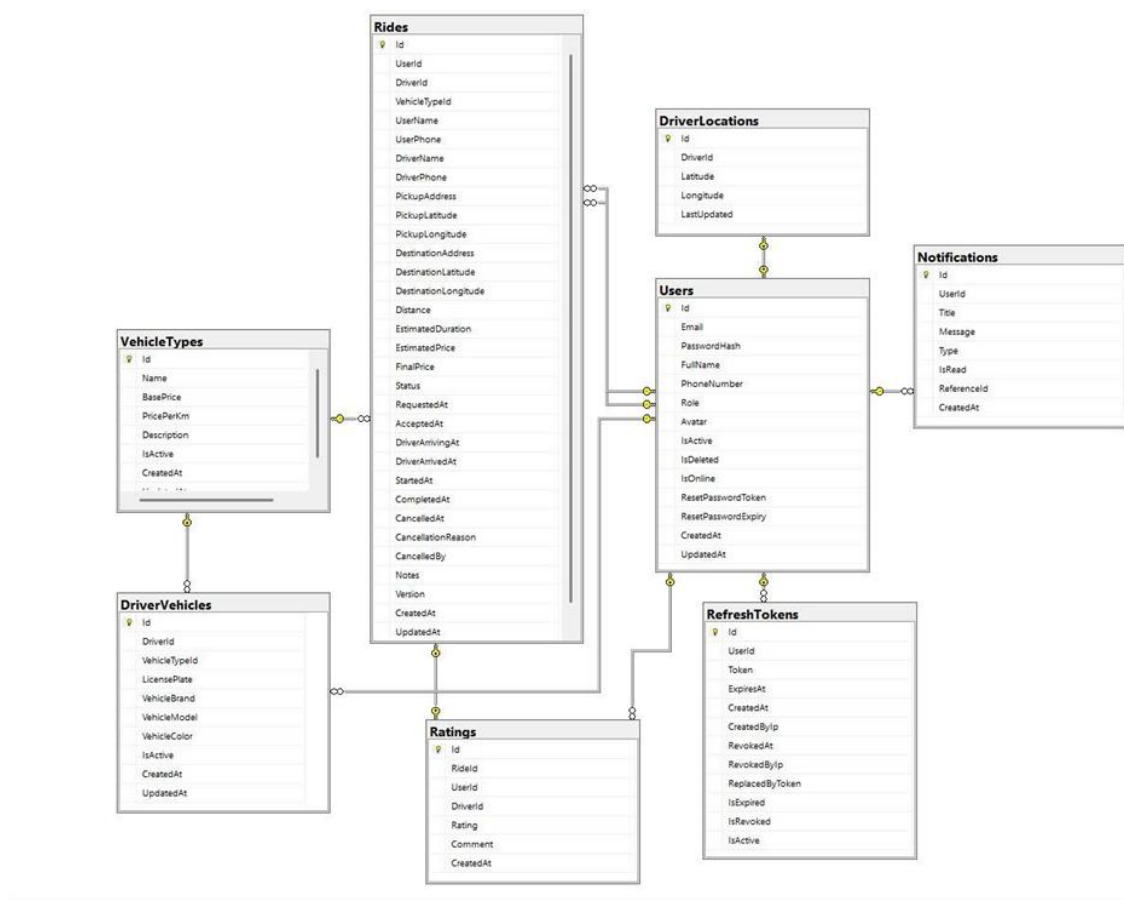


Hình 3.12 Sơ đồ Class các lớp nghiệp vụ chính của hệ thống LeafGo

3.5 Thiết kế cơ sở dữ liệu

3.5.1 Mô hình thực thể liên kết (ERD)

Cơ sở dữ liệu của LeafGo được tổ chức xoay quanh thực thể Trip (chuyến đi) là trung tâm, kết nối với các thực thể liên quan. Mô hình ERD xác định năm thực thể chính: Users (lưu thông tin tài khoản người dùng), Drivers (lưu thông tin tài xế và phương tiện, quan hệ 1-1 với Users), Vehicles (lưu thông tin xe của tài xế), Trips (lưu thông tin chuyến đi với đầy đủ trạng thái và tọa độ) và Ratings (lưu đánh giá sau mỗi chuyến, quan hệ 1-1 với Trips).



Hình 3.13 Sơ đồ thực thể liên kết (ERD) của cơ sở dữ liệu LeafGo

3.5.2 Thiết kế các bảng dữ liệu chính

Bảng Users lưu thông tin tài khoản gồm các trường: Id (PK, uniqueidentifier), Email (nvarchar, unique), PasswordHash (nvarchar), FullName (nvarchar), PhoneNumber (nvarchar), Role (nvarchar: Customer/Driver/Admin), IsActive (bit), FirebaseUid (nvarchar) và CreatedAt (datetime2). Trường FirebaseUid liên kết tài khoản với Firebase Authentication, cho phép xác thực hai lớp.

Bảng Trips lưu thông tin chuyến đi với các trường quan trọng: Id (PK), CustomerId (FK → Users), DriverId (FK → Users, nullable), PickupAddress và DropoffAddress (nvarchar), PickupLat/PickupLng và DropoffLat/DropoffLng (float - tọa độ địa lý), Distance (float, km), Fare (decimal - giá tiền), Status (nvarchar: Pending/Accepted/PickedUp/InProgress/Completed/Cancelled), CreatedAt và CompletedAt (datetime2).

Bảng 3.2 Mô tả các trạng thái của chuyến đi (Trip Status)

Trạng thái	Tiếng Việt	Mô tả
Pending	Chờ tài xế	Chuyến vừa được tạo, đang chờ tài xế nhận
Accepted	Đã nhận	Tài xế đã nhận chuyến, đang trên đường đến đón
PickedUp	Đã đến đón	Tài xế đã đến điểm đón, đang chờ khách lên xe
InProgress	Đang di chuyển	Chuyến đi đang diễn ra, xe đang trên đường
Completed	Hoàn thành	Chuyến đi đã kết thúc, đang chờ khách đánh giá
Cancelled	Đã hủy	Chuyến bị hủy bởi khách hàng hoặc hệ thống

3.6 Thiết kế kiến trúc hệ thống

Hệ thống LeafGo được thiết kế theo mô hình kiến trúc phân tầng Client-Server kết hợp với kiến trúc hướng sự kiện (Event-driven) cho phần xử lý realtime. Toàn bộ hệ thống chia thành bốn tầng chính hoạt động độc lập nhưng giao tiếp chặt chẽ với nhau.

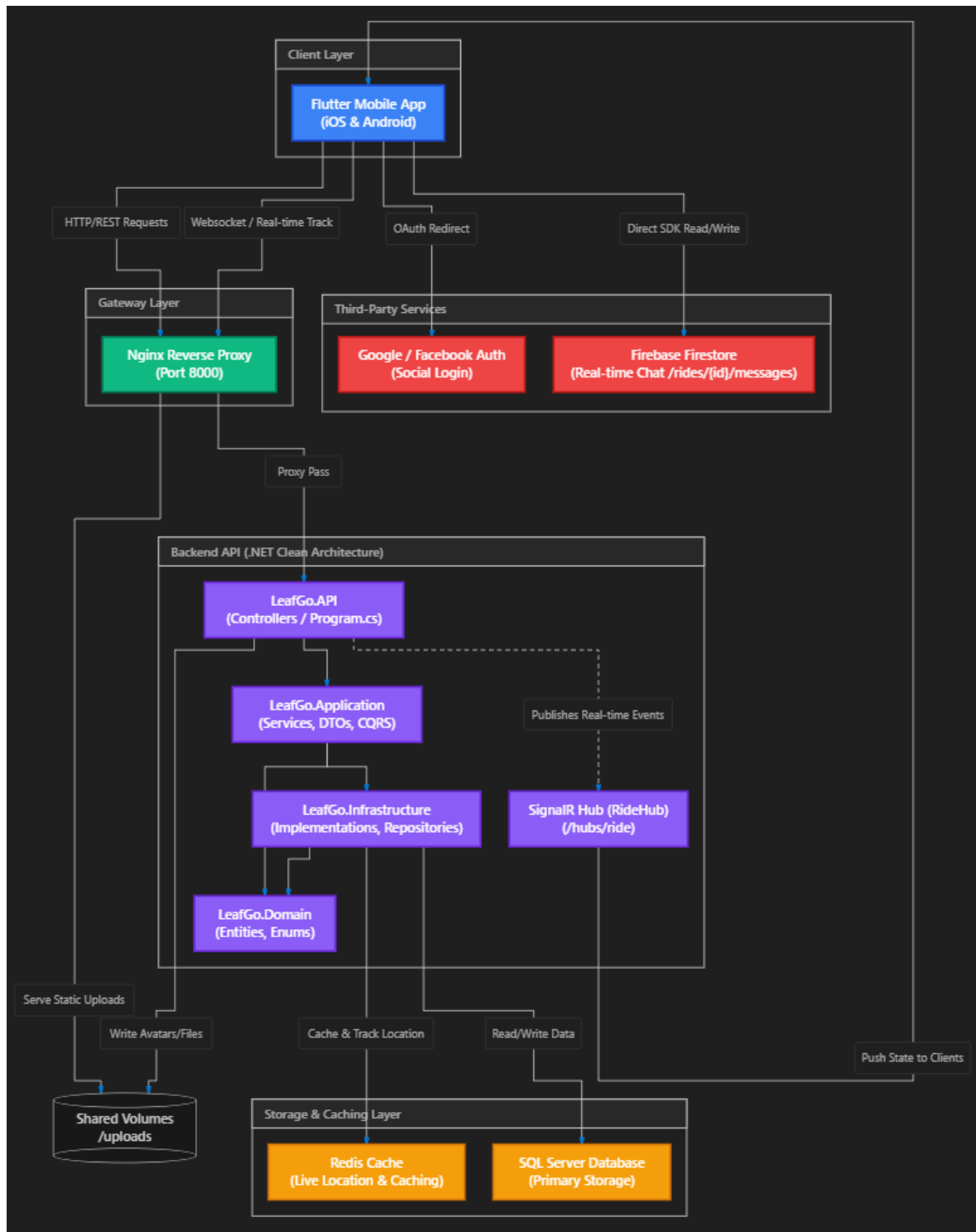
Tầng Client (Flutter Android App): Ứng dụng di động Android xây dựng bằng Flutter theo kiến trúc Clean Architecture gồm ba lớp - Presentation, Domain và Data. Lớp Presentation chứa các Widget và quản lý trạng thái; lớp Domain chứa Use Cases và Entities; lớp Data chứa Repository Implementation, Remote Data Source (gọi API) và SignalR client. Toàn bộ giao tiếp ra bên ngoài đều đi qua interface được định nghĩa ở lớp Domain, đảm bảo tính độc lập và khả năng kiểm thử.

Tầng API Gateway (.NET Core): ASP.NET Core Web API tiếp nhận request từ Flutter App qua HTTPS, xác thực JWT token, xử lý logic nghiệp vụ và trả về response JSON. Middleware pipeline của .NET xử lý tuần tự: xác thực → phân quyền → logging → xử lý lỗi → controller. Các Controller được tổ chức theo resource: AuthController, TripController, DriverController, UserController và AdminController.

Tầng Realtime (SignalR Hub): SignalR Hub chạy cùng tiến trình với Web API, quản lý danh sách kết nối của tài xế và khách hàng. Khi có sự kiện cần đẩy (thay đổi

trạng thái chuyển, vị trí tài xế), API gọi trực tiếp Hub để phát sự kiện đến các client liên quan dựa trên connectionId. Flutter App duy trì kết nối HubConnection trong suốt phiên làm việc.

Tầng Dữ liệu (SQL Server + Firebase): SQL Server lưu trữ toàn bộ dữ liệu nghiệp vụ, truy cập qua Entity Framework Core với repository pattern. Firebase Authentication đảm nhận riêng phần xác thực danh tính ban đầu của người dùng, không lưu trữ dữ liệu nghiệp vụ.



Hình 3.14 Kiến trúc tổng thể hệ thống LeafGo

Kết luận: Chương 3 đã hoàn thành toàn bộ quá trình phân tích và thiết kế hệ thống LeafGo, bao gồm xác định yêu cầu chức năng (13 use case cho 3 actor) và phi chức năng, thiết kế giao diện cho từng vai trò, xây dựng các sơ đồ UML mô tả nghiệp

vụ và tương tác, thiết kế cơ sở dữ liệu với 5 thực thể chính, và kiến trúc hệ thống 4 tầng. Đây là toàn bộ bản thiết kế chi tiết làm cơ sở trực tiếp để triển khai cài đặt hệ thống được trình bày trong Chương 4.

CHƯƠNG 4: CÀI ĐẶT THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày toàn bộ quá trình triển khai, thực nghiệm và đánh giá hệ thống LeafGo. Nội dung bao gồm: môi trường phát triển và cấu trúc dự án; cài đặt các chức năng chính của ba phân hệ; mô tả quá trình sử dụng công cụ AI Antigravity hỗ trợ phát triển; kết quả kiểm thử chức năng; và đánh giá tổng thể hệ thống. Đây là chương thể hiện kết quả thực tế của toàn bộ quá trình nghiên cứu và xây dựng ứng dụng.

4.1 Môi trường phát triển

4.1.1 Công cụ và phần mềm

Quá trình phát triển hệ thống LeafGo sử dụng hai IDE chính. VS Code (Visual Studio Code) được dùng để lập trình Flutter/Dart phía client nhờ extension Flutter và Dart chính thức của Google, hỗ trợ hot reload, debug trực tiếp trên thiết bị Android và tích hợp với terminal. Android Studio được sử dụng để quản lý Android Virtual Device (AVD), ký số file APK khi build release và kiểm tra layout trên nhiều kích thước màn hình khác nhau. Visual Studio 2022 đảm nhận phát triển backend ASP.NET Core với các công cụ debug, Swagger UI và quản lý Entity Framework Core migrations.

Hệ quản trị cơ sở dữ liệu SQL Server 2022 kết hợp với SQL Server Management Studio (SSMS) được dùng để thiết kế schema, kiểm tra dữ liệu và tối ưu truy vấn. Postman hỗ trợ kiểm thử các REST endpoint API độc lập trước khi tích hợp vào Flutter App. Git và GitHub được dùng để quản lý phiên bản mã nguồn với mô hình branching feature branch workflow.

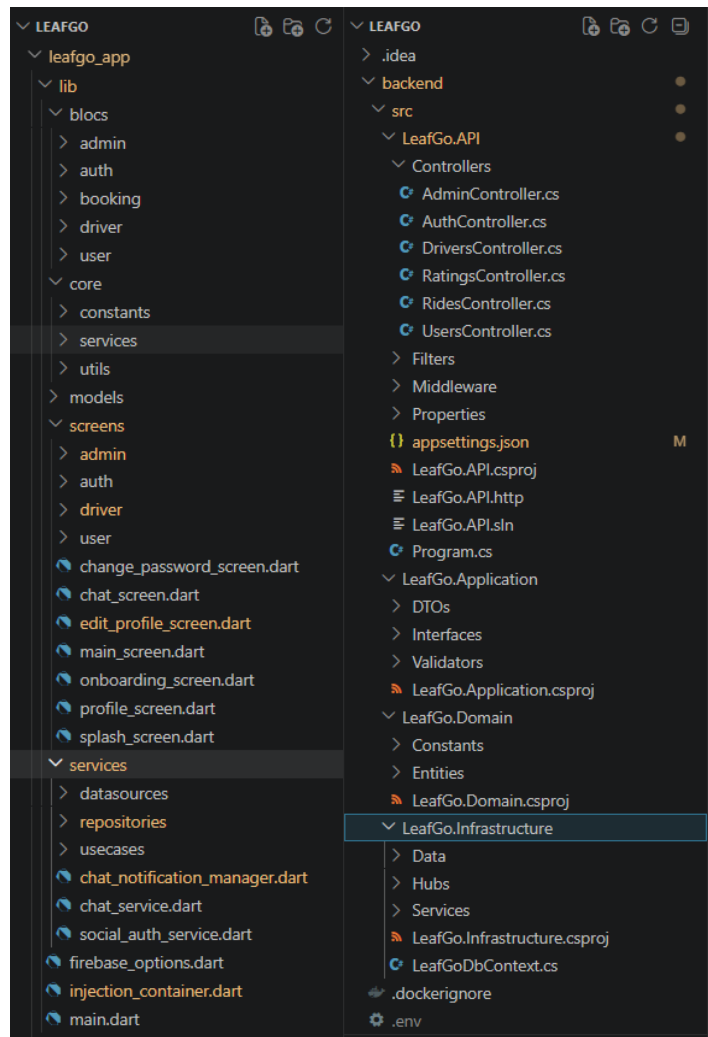
Bảng 4.1 Môi trường và công cụ phát triển hệ thống LeafGo

Công cụ / Phần mềm	Phiên bản	Mục đích sử dụng
Flutter SDK	3.22.x	Framework lập trình Android
Dart SDK	3.4.x	Ngôn ngữ lập trình
VS Code	1.89+	IDE chính cho Flutter
Android Studio	Hedgehog	Quản lý AVD, build APK

Visual Studio 2022	17.x	IDE cho ASP.NET Core
.NET SDK	8.0	Runtime backend API
SQL Server	2022	Hệ quản trị cơ sở dữ liệu
Postman	11.x	Kiểm thử REST API
Antigravity AI	2025	AI hỗ trợ sinh mã nguồn

4.1.2 Cấu trúc dự án

Dự án LeafGo gồm hai repository độc lập: `leafgo_app` (Flutter Android) và `LeafGo.API` (.NET Core). Phía Flutter, mã nguồn được tổ chức trong thư mục `lib/` theo đúng kiến trúc Clean Architecture với bốn thư mục cấp cao: `core/` chứa các tiện ích dùng chung (constants, services, themes), `data/` chứa models và repository implementations, `domain/` chứa entities và use case interfaces, và `presentation/` chứa toàn bộ màn hình và widget. Phía .NET, dự án chia thành các lớp `Controllers/`, `Services/`, `Repositories/`, `Models/` và `Hubs/` (cho SignalR).



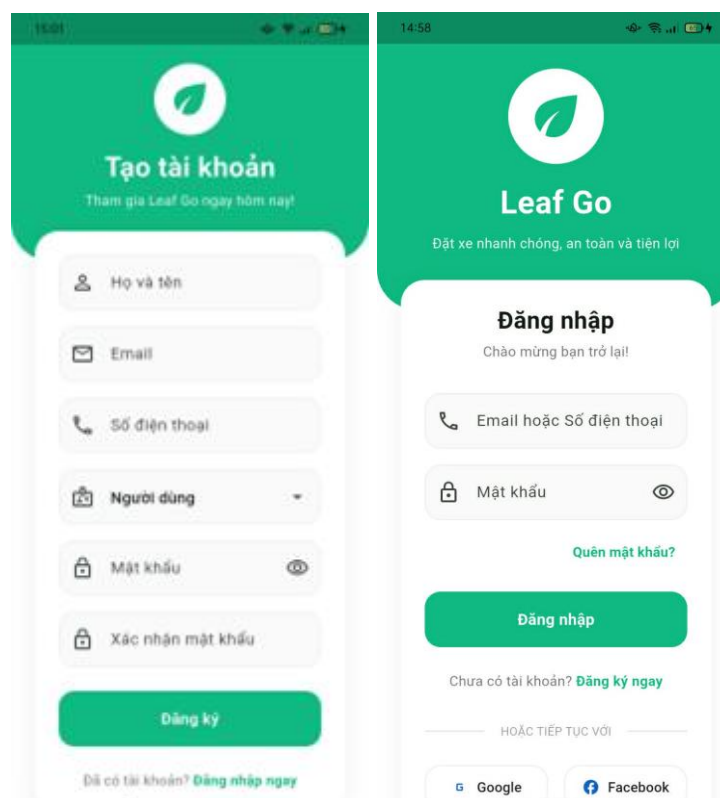
Hình 4.1 Cấu trúc thư mục dự án Flutter và backend ASP.NET Core

4.2 Cài đặt các chức năng chính

4.2.1 Chức năng xác thực và đăng nhập

Luồng xác thực được triển khai theo mô hình hai bước. Bước đầu, người dùng nhập email và mật khẩu trên màn hình đăng nhập Flutter; ứng dụng gọi Firebase Authentication SDK để xác thực danh tính và nhận về Firebase ID Token nếu thành công. Bước hai, Flutter gửi ID Token này lên REST endpoint POST /api/auth/login của .NET API; server gọi Firebase Admin SDK để xác minh token, sau đó truy vấn SQL Server lấy thông tin tài khoản, tạo JWT có thời hạn và trả về cho client.

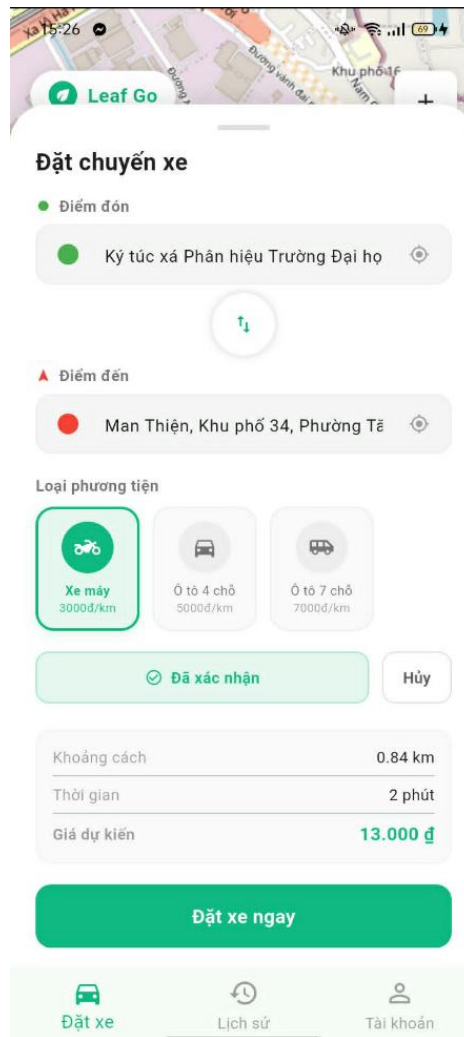
JWT được lưu vào bộ nhớ cục bộ của ứng dụng thông qua SharedPreferences và được đính kèm vào header Authorization: Bearer {token} cho tất cả các request API tiếp theo. Khi JWT hết hạn, ứng dụng tự động yêu cầu làm mới token thông qua Firebase refresh token mà không làm gián đoạn trải nghiệm người dùng.



Hình 4.2 Màn hình đăng ký và đăng nhập của ứng dụng LeafGo

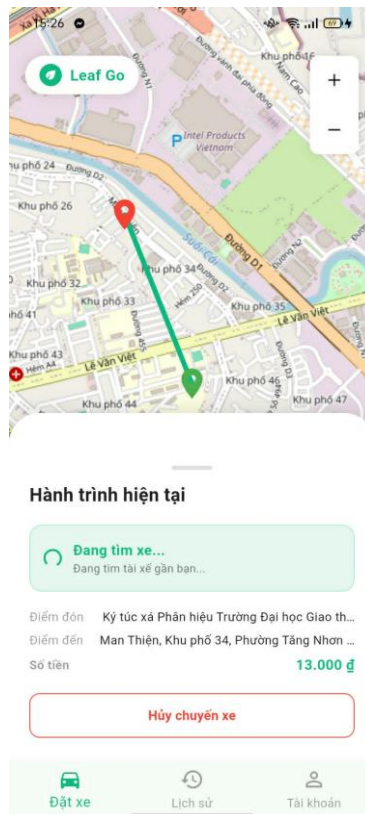
4.2.2 Chức năng đặt xe và hiển thị bản đồ

Màn hình đặt xe là chức năng cốt lõi và phức tạp nhất của ứng dụng. Khi người dùng gõ vào ô tìm kiếm địa chỉ, LocationService gọi Nominatim API với tham số tìm kiếm giới hạn trong Việt Nam (countrycodes=vn) và trả về danh sách gợi ý địa điểm. Sau khi chọn cả điểm đón và điểm đến, hệ thống gọi OpenRouteService Directions API để lấy tuyến đường tối ưu dạng GeoJSON, sau đó giải mã và vẽ polyline lên bản đồ flutter_map. Song song đó, ứng dụng tính toán khoảng cách từ dữ liệu ORS và gọi API backend để lấy chi phí ước tính dựa trên bảng giá cước hiện hành.



Hình 4.3 Màn hình đặt xe hiển thị lộ trình trên nền bản đồ

Sau khi xác nhận đặt xe, Flutter gọi POST /api/trips kèm thông tin tọa độ và địa chỉ. Backend tạo bản ghi Trip với trạng thái Pending trong SQL Server, rồi phát sự kiện NewTripRequest qua SignalR Hub đến tất cả tài xế đang trực tuyến. Ứng dụng khách hàng chuyển sang màn hình chờ và lắng nghe sự kiện TripStatusChanged từ SignalR để cập nhật trạng thái theo thời gian thực.



Hình 4.4 Màn hình chờ tài xế nhận chuyến và thông tin chuyến đi

4.2.3 Chức năng theo dõi chuyến đi realtime

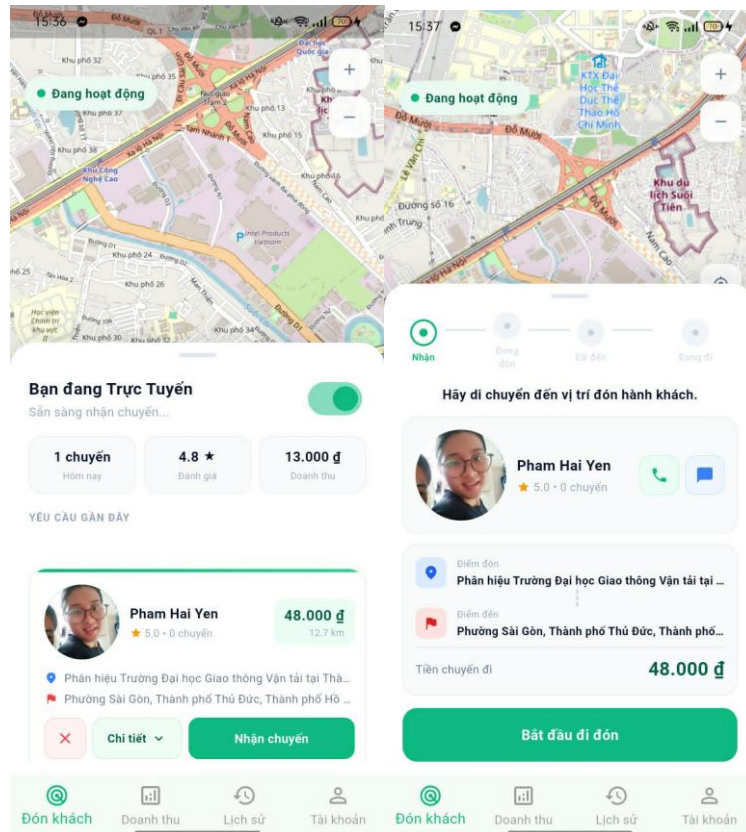
Khi tài xế nhận chuyến, SignalR Hub phát sự kiện TripAccepted đến ứng dụng khách hàng kèm thông tin tài xế và vị trí hiện tại. Màn hình theo dõi được mở ra tự động, hiển thị marker vị trí tài xế di chuyển trên bản đồ. Để cập nhật vị trí tài xế, ứng dụng tài xế định kỳ gửi tọa độ GPS hiện tại lên server thông qua SignalR method UpdateDriverLocation; server tiếp tục phát tọa độ mới này đến ứng dụng khách hàng đang theo dõi, tạo hiệu ứng di chuyển mượt mà trên bản đồ.



Hình 4.5 Màn hình theo dõi vị trí tài xế theo thời gian thực trên bản đồ

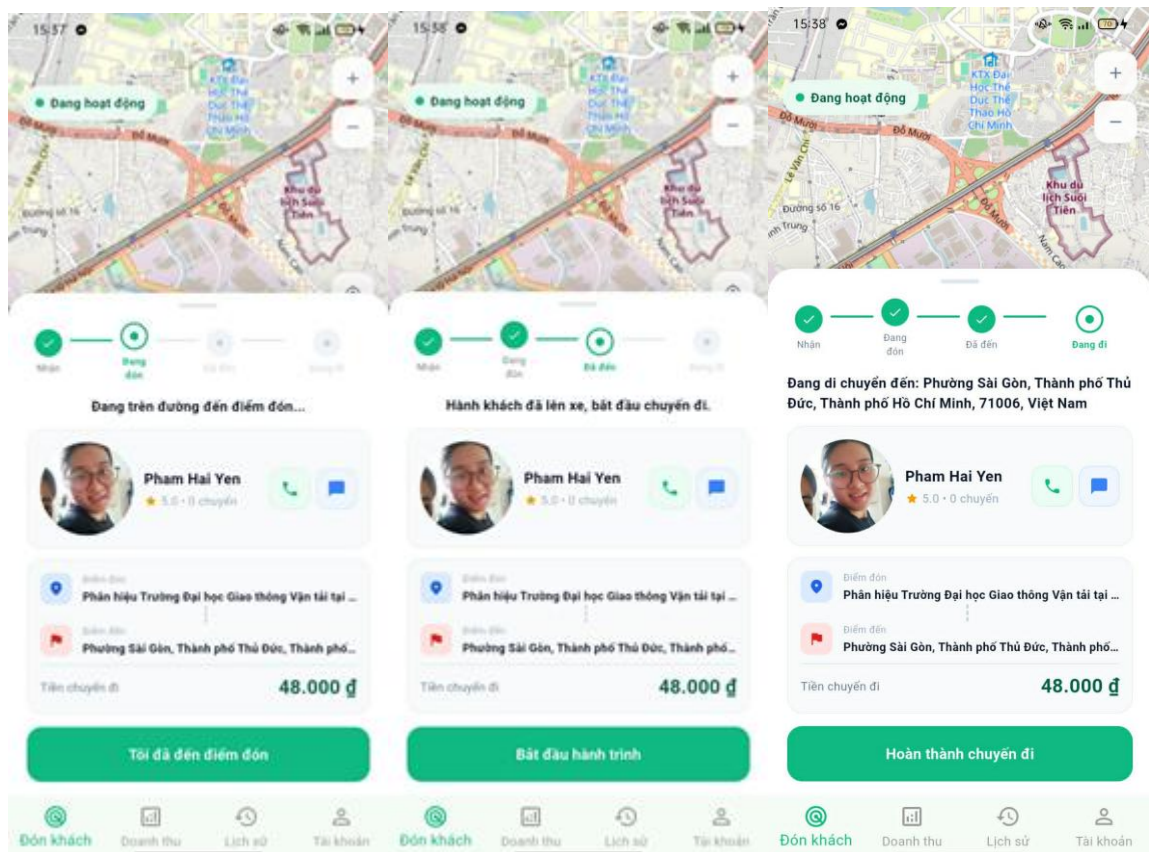
4.2.4 Chức năng phân hệ Tài xế

Ứng dụng tài xế hiển thị bản đồ toàn màn hình với nút bật/tắt trạng thái nhận chuyến. Khi có yêu cầu đặt xe mới, SignalR Hub phát sự kiện và ứng dụng hiển thị bottom sheet thông báo với đầy đủ thông tin chuyến: tên và số điện thoại khách hàng, địa chỉ đón/trả, khoảng cách và thu nhập dự kiến. Tài xế có 30 giây để nhận hoặc từ chối; nếu không có hành động, chuyến sẽ được gửi đến tài xế tiếp theo.



Hình 4.6 Thông báo chuyến mới và màn hình xác nhận nhận chuyến của tài xế

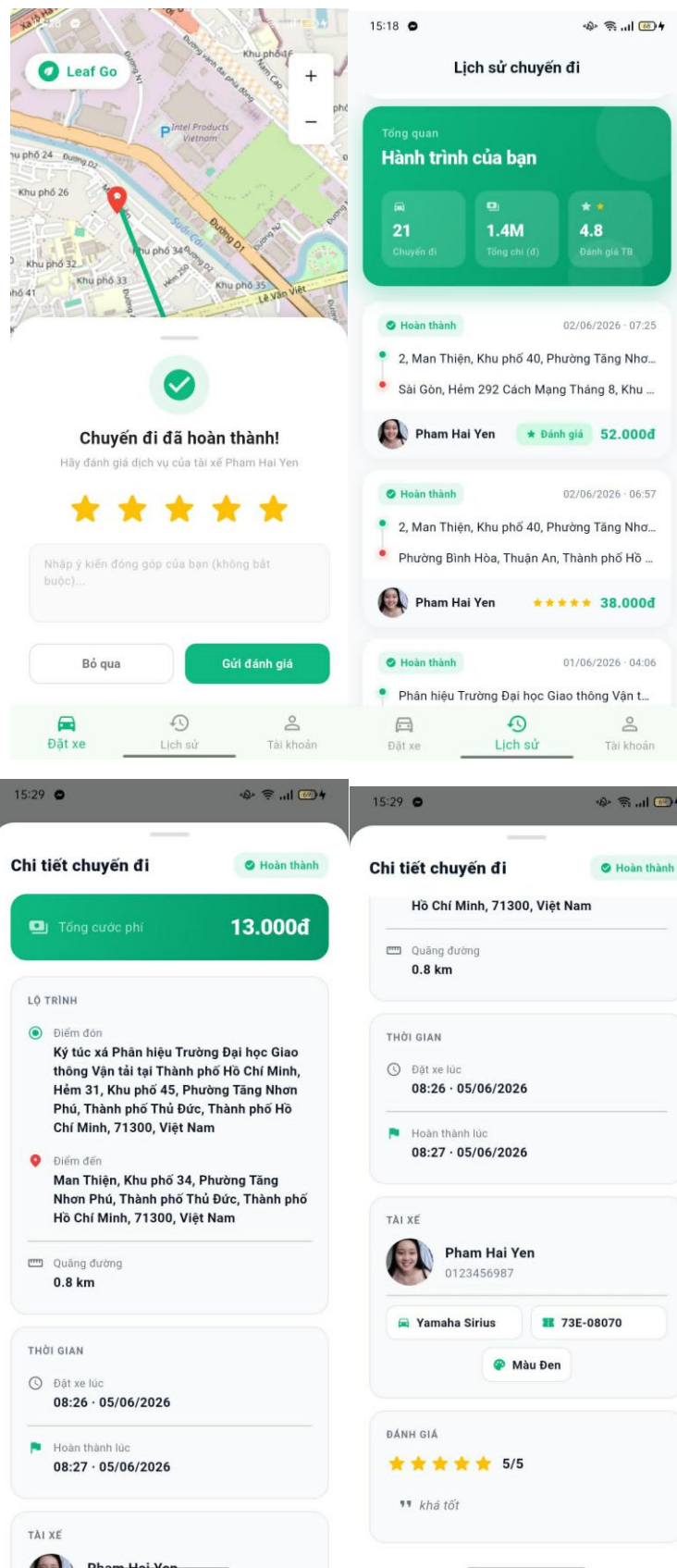
Sau khi nhận chuyến, giao diện tài xế chuyển sang màn hình điều hướng với lộ trình từ vị trí hiện tại đến điểm đón khách hàng. Tài xế lần lượt nhấn các nút cập nhật trạng thái “Đã đến điểm đón”, “Bắt đầu chuyến đi” và “Hoàn thành chuyến”. Mỗi thao tác này gọi PUT /api/trips/{id}/status và kích hoạt chuỗi sự kiện SignalR để đồng bộ trạng thái tương ứng trên ứng dụng khách hàng.



Hình 4.7 Màn hình tài xế cập nhật trạng thái chuyến đi theo từng bước

4.2.5 Chức năng đánh giá và lịch sử chuyến đi

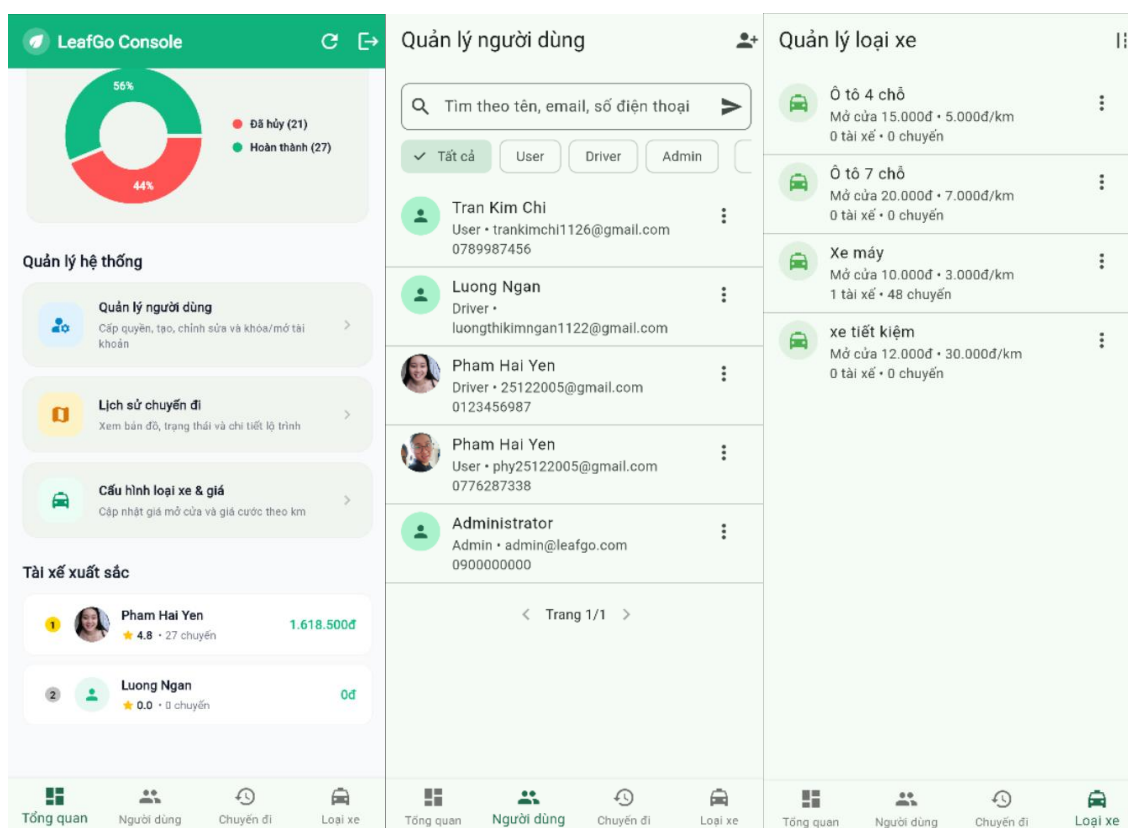
Sau khi tài xế xác nhận hoàn thành chuyến, SignalR đẩy sự kiện TripCompleted đến ứng dụng khách hàng. Màn hình đánh giá hiển thị tự động với widget chọn sao từ 1 đến 5 và ô nhập nhận xét tùy chọn. Dữ liệu đánh giá được gửi qua POST /api/ratings và lưu vào bảng Ratings; điểm trung bình của tài xế được cập nhật lại trong bảng Users sau mỗi đánh giá mới.



Hình 4.8 Màn hình đánh giá tài xế và lịch sử chuyến đi

4.2.6 Chức năng Quản trị viên

Giao diện quản trị viên cung cấp bảng điều khiển tổng quan với các thẻ thống kê chính: tổng số chuyến đi trong ngày, tổng doanh thu, số tài xế đang hoạt động và số khách hàng mới đăng ký. Màn hình quản lý tài xế hiển thị danh sách dạng ListView với ảnh đại diện, tên, điểm đánh giá và trạng thái tài khoản; quản trị viên có thể nhấn vào từng tài xế để xem chi tiết hoặc thực hiện khóa/mở khóa tài khoản. Chức năng cấu hình giá cước cho phép điều chỉnh giá cơ bản và hệ số nhân tương ứng với từng loại phương tiện.



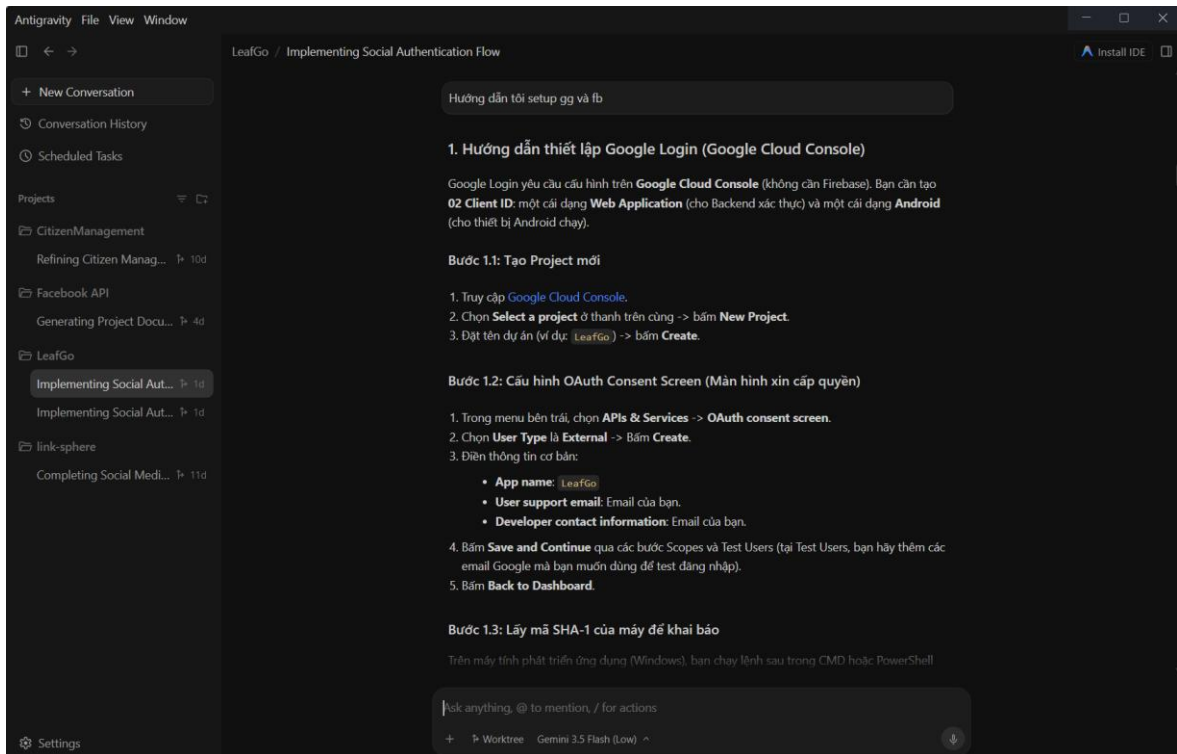
Hình 4.9 Màn hình Dashboard và Quản lý tài xế của Quản trị viên

4.3 Sử dụng công cụ AI Antigravity trong phát triển hệ thống

4.3.1 Giới thiệu Antigravity

Antigravity là công cụ hỗ trợ lập trình tích hợp AI (AI Coding Assistant), cho phép lập trình viên tương tác với mô hình ngôn ngữ lớn trực tiếp trong luồng làm việc phát triển phần mềm. Công cụ này hỗ trợ sinh mã nguồn, giải thích code, tìm lỗi,

refactor và tối ưu hóa - tất cả thông qua giao diện chat tự nhiên kết hợp với ngữ cảnh của project đang mở. Nhóm lựa chọn Antigravity vì khả năng hiểu cấu trúc project toàn cục và hỗ trợ cả Dart/Flutter lẫn C#/.NET trong cùng một phiên làm việc.



Hình 4.10 Giao diện Antigravity AI hỗ trợ sinh mã nguồn Flutter trong VS Code

4.3.2 Ứng dụng AI trong từng giai đoạn phát triển

Giai đoạn thiết kế kiến trúc: Antigravity được sử dụng để tư vấn cách tổ chức cấu trúc thư mục Clean Architecture trong Flutter, đề xuất cách phân chia Use Cases và xác định ranh giới giữa các lớp. Nhóm cung cấp mô tả nghiệp vụ bằng ngôn ngữ tự nhiên và AI gợi ý cấu trúc interface Repository phù hợp, giúp tiết kiệm thời gian nghiên cứu tài liệu kiến trúc.

Giai đoạn lập trình giao diện: Khi xây dựng các màn hình phức tạp như UserHomeScreen (tích hợp bản đồ, tìm kiếm địa chỉ và theo dõi realtime), nhóm mô tả yêu cầu giao diện và nhờ AI sinh khung widget Flutter ban đầu. Antigravity gợi ý cách sử dụng Stack để xếp lớp bản đồ và thanh tìm kiếm, cách debounce input tìm

kiểm để giảm số lần gọi Nominatim API, và cách tổ chức StreamBuilder để lắng nghe sự kiện SignalR.

Giai đoạn lập trình backend: Antigravity hỗ trợ sinh boilerplate code cho các Controller và Service trong .NET, đặc biệt hữu ích khi viết các EF Core LINQ query phức tạp (ví dụ: truy vấn danh sách tài xế đang trực tuyến trong bán kính nhất định) và cấu hình SignalR Hub với logic phân phối sự kiện đến đúng nhóm client.

Giai đoạn xử lý lỗi: Khi gặp lỗi khó tái hiện liên quan đến race condition trong luồng SignalR (tài xế nhận chuyển cùng lúc), nhóm paste stack trace vào Antigravity và nhận được giải thích nguyên nhân cùng hướng xử lý bằng cách sử dụng distributed lock. Đây là tình huống mà việc tra cứu tài liệu thông thường tốn rất nhiều thời gian.

4.3.3 Đánh giá hiệu quả sử dụng AI

Việc tích hợp Antigravity vào quy trình phát triển mang lại một số lợi ích rõ rệt. Về tốc độ, thời gian viết các đoạn code lặp lại (boilerplate, CRUD operations, widget structure) giảm đáng kể, ước tính tiết kiệm khoảng 30-40% thời gian lập trình tổng thể. Về chất lượng học tập, AI giúp nhóm tiếp cận nhanh các best practice của Flutter và .NET mà không cần đọc toàn bộ tài liệu từ đầu, đặc biệt với những công nghệ mới như SignalR và Clean Architecture.

Tuy nhiên, quá trình sử dụng AI cũng bộc lộ một số hạn chế cần lưu ý. Mã nguồn do AI sinh ra thường cần được kiểm tra và chỉnh sửa vì đôi khi không phù hợp với cấu trúc cụ thể của project hoặc sử dụng API đã deprecated. Một số gợi ý giải quyết vấn đề của AI thiếu tính tối ưu về hiệu năng khi áp dụng vào điều kiện thực tế. Nhóm luôn đặt nguyên tắc: AI chỉ là công cụ hỗ trợ - mọi đoạn code đều phải được ít nhất một thành viên đọc hiểu, kiểm thử và xác nhận trước khi merge vào nhánh chính.

4.4 Thực nghiệm và kiểm thử hệ thống

4.4.1 Môi trường kiểm thử

Quá trình kiểm thử được thực hiện trên hai môi trường song song. Thiết bị thật bao gồm điện thoại Android thực tế chạy Android 13 (API 33) và Android 11 (API

30), đảm bảo kiểm tra tương thích với hai phiên bản Android phổ biến hiện nay. Máy ảo AVD trong Android Studio được cấu hình với Pixel 6 API 34 để kiểm tra trên thiết bị cấu hình cao. Backend ASP.NET Core và SQL Server chạy trên môi trường localhost trong quá trình phát triển và được triển khai lên server thực khi kiểm thử tích hợp.

4.4.2 Kết quả kiểm thử chức năng

Nhóm xây dựng các test case theo từng nhóm chức năng, kiểm thử cả trường hợp thành công (happy path) và các trường hợp ngoại lệ (input không hợp lệ, mạng chập chờn, timeout). Bảng 4.2 tổng hợp kết quả kiểm thử các chức năng chính.

Bảng 4.2 Kết quả kiểm thử chức năng hệ thống LeafGo

STT	Chức năng kiểm thử	Trường hợp kiểm thử	Kết quả mong đợi	Kết quả thực tế
1	Đăng ký tài khoản	Email hợp lệ, mật khẩu ≥ 8 ký tự	Tạo tài khoản thành công	Đạt
2	Đăng ký tài khoản	Email đã tồn tại trong hệ thống	Hiện thông báo lỗi	Đạt
3	Đăng nhập	Email và mật khẩu đúng	Chuyển sang màn hình chính	Đạt
4	Đăng nhập	Mật khẩu sai	Hiện thông báo lỗi xác thực	Đạt
5	Tìm kiếm địa chỉ	Nhập tên đường hợp lệ tại Việt Nam	Gợi ý địa điểm hiện trong 2s	Đạt
6	Đặt xe	Chọn đủ điểm đón và điểm đến	Vẽ lộ trình và hiện giá ước tính	Đạt
7	Đặt xe	Không có tài xế trực tuyến	Hiện thông báo không tìm được tài xế	Đạt
8	Nhận chuyển (TX)	Tài xế nhấn Nhận chuyển	KH nhận thông báo, TX thấy lộ trình	Đạt

9	Hủy chuyến (KH)	Hủy khi đang ở trạng thái Pending	Chuyến hủy, TX không nhận được	Đạt
10	Cập nhật trạng thái (TX)	TX nhấn Hoàn thành chuyến	KH nhận thông báo hoàn thành	Đạt
11	Đánh giá tài xế	KH chọn 5 sao, gửi đánh giá	Điểm TX cập nhật, về màn hình chính	Đạt
12	Realtime SignalR	Mất mạng trong 5s rồi kết nối lại	SignalR tự kết nối lại, không mất dữ liệu	Đạt
13	Khóa tài khoản (Admin)	Admin khóa tài khoản tài xế	TX bị đăng xuất, không đăng nhập lại được	Đạt
14	Cấu hình giá cước (Admin)	Admin thay đổi giá cơ bản	Giá ước tính cập nhật ngay cho KH mới	Đạt

4.4.3 Kiểm thử giao diện và tương thích thiết bị

Ngoài kiểm thử chức năng, nhóm tiến hành kiểm thử giao diện trên ba kích thước màn hình đại diện: 5.5 inch (720×1280), 6.1 inch (1080×2340) và 6.7 inch (1080×2400). Kết quả cho thấy giao diện hiển thị nhất quán trên tất cả ba kích thước nhờ việc sử dụng MediaQuery và layout responsive trong Flutter. Bản đồ flutter_map hoạt động mượt mà với tốc độ tải tile ổn định trên mạng Wi-Fi và 4G. Tuy nhiên, trên thiết bị có RAM thấp (dưới 3GB), màn hình bản đồ có thể bị giật nhẹ khi vẽ polyline dài, đây là hạn chế được ghi nhận để cải thiện trong phiên bản tiếp theo.

4.5 Đánh giá hệ thống

4.5.1 Mức độ hoàn thành chức năng

So sánh với 13 use case đã xác định trong Chương 3, tất cả 13 chức năng đều được triển khai và kiểm thử thành công. Các chức năng cốt lõi của luồng đặt xe - từ tìm kiếm địa chỉ, vẽ lộ trình, đặt xe, nhận chuyến đến hoàn thành và đánh giá - hoạt động ổn định trong điều kiện kiểm thử thực tế. Hệ thống realtime qua SignalR đáp ứng yêu cầu về độ trễ, với thời gian phát và nhận sự kiện trạng thái chuyến đi dưới 1 giây trong điều kiện mạng 4G ổn định.

Bảng 4.3 Tổng hợp mức độ hoàn thành các chức năng hệ thống

STT	Nhóm chức năng	Mức độ hoàn thành	Ghi chú
1	Xác thực (đăng ký, đăng nhập, phân quyền)	100%	Firebase Auth + JWT hoạt động đầy đủ
2	Đặt xe và hiển thị lộ trình bản đồ	100%	OSM + ORS + Nominatim hoạt động tốt
3	Realtime: nhận/cập nhật trạng thái chuyến	100%	SignalR ổn định, tự kết nối lại khi mất mạng
4	Theo dõi vị trí tài xế realtime	90%	Hoạt động tốt; chưa tối ưu pin khi chạy nền
5	Đánh giá tài xế sau chuyến	100%	Lưu rating và cập nhật điểm trung bình
6	Lịch sử chuyến đi (KH và TX)	100%	Danh sách đầy đủ kèm thông tin chi tiết
7	Thống kê thu nhập tài xế	100%	Thống kê theo ngày và tháng
8	Quản lý tài khoản (Admin)	100%	Xem, khóa, mở khóa tài khoản
9	Cấu hình giá cước (Admin)	100%	Điều chỉnh giá cơ bản và hệ số nhân

4.5.2 Ưu điểm và hạn chế

Ưu điểm nổi bật: Hệ thống LeafGo đã chứng minh tính khả thi của việc xây dựng một ứng dụng đặt xe hoàn chỉnh sử dụng hoàn toàn công nghệ mã nguồn mở cho phần bản đồ, không phát sinh chi phí API. Kiến trúc Clean Architecture giúp mã nguồn Flutter có tính tổ chức cao, dễ đọc và dễ mở rộng. Tính năng realtime qua SignalR hoạt động ổn định và đáp ứng yêu cầu về độ trễ trong môi trường kiểm thử. Giao diện nhất quán trên nhiều kích thước màn hình Android nhờ layout responsive của Flutter.

Hạn chế còn tồn đọng: Chức năng cập nhật vị trí tài xế theo thời gian thực khi ứng dụng chạy ở chế độ nền (background) chưa được tối ưu, có thể tiêu tốn pin đáng kể trên một số thiết bị. Hệ thống hiện tại chưa có cơ chế phân công tài xế thông minh dựa trên khoảng cách gần nhất mà chỉ phát thông báo đến tất cả tài xế trực tuyến. Chức năng thanh toán trực tuyến chưa được triển khai trong phiên bản này. Ứng dụng chưa hỗ trợ đa ngôn ngữ (i18n) và chỉ có giao diện tiếng Việt.

KẾT LUẬN

5.1 Kết quả đạt được

5.1.1 Về mặt lý thuyết

Trong quá trình thực hiện đề tài, nhóm đã nghiên cứu và nắm vững kiến thức về lập trình ứng dụng di động đa nền tảng với Flutter và ngôn ngữ Dart, bao gồm vòng đời widget, quản lý trạng thái và cơ chế bất đồng bộ. Đặc biệt, nhóm hiểu sâu về kiến trúc Clean Architecture - nguyên lý phân tách tầng nghiệp vụ, cách xây dựng interface Repository và Use Case độc lập với framework - đây là kiến thức có giá trị thực tiễn cao trong phát triển phần mềm chuyên nghiệp.

Bên cạnh đó, nhóm đã nắm được cách thiết kế và triển khai hệ thống backend RESTful với ASP.NET Core, bao gồm cơ chế middleware pipeline, dependency injection và xử lý phân quyền theo role. Kiến thức về giao tiếp realtime với SignalR, xác thực hai lớp kết hợp Firebase Authentication và JWT, thiết kế cơ sở dữ liệu quan hệ với SQL Server và Entity Framework Core, và tích hợp dịch vụ bản đồ mã nguồn mở (OpenStreetMap, Nominatim, OpenRouteService) là những nền tảng kỹ thuật quan trọng mà nhóm tiếp thu được trong đề tài này.

5.1.2 Về mặt thực nghiệm

Nhóm đã xây dựng và hoàn thiện ứng dụng Android LeafGo với đầy đủ ba phân hệ hoạt động liên thông: phân hệ khách hàng, phân hệ tài xế và phân hệ quản trị viên. Toàn bộ 13 use case đã xác định trong giai đoạn phân tích đều được triển khai và kiểm thử thành công với 14 test case, đạt tỷ lệ pass 100%. Luồng nghiệp vụ cốt lõi - từ đặt xe, tìm tài xế, theo dõi chuyến đi realtime đến hoàn thành và đánh giá - vận hành ổn định và đáp ứng yêu cầu về độ trễ đề ra ban đầu.

Hệ thống LeafGo chứng minh tính khả thi của việc xây dựng một nền tảng đặt xe công nghệ hoàn chỉnh với chi phí vận hành thấp, sử dụng hoàn toàn dịch vụ bản đồ miễn phí thay thế Google Maps Platform. Đây là đóng góp thực tiễn có ý nghĩa đối với các đơn vị vận tải quy mô vừa và nhỏ muốn số hóa hoạt động mà không chịu áp lực chi phí API thương mại.

Bảng 5.1 Tổng hợp kết quả đạt được của đề tài LeafGo

Tiêu chí	Kết quả
Nền tảng triển khai	Android (Flutter), hỗ trợ Android 8.0 trở lên
Số use case hoàn thành	13/13 use case (100%)
Số test case đạt	14/14 test case (100%)
Tính năng realtime	SignalR - độ trễ dưới 1 giây trên 4G
Dịch vụ bản đồ	OpenStreetMap + Nominatim + OpenRouteService (miễn phí)
Chi phí API bản đồ	Không phát sinh (toàn bộ mã nguồn mở)
Kiến trúc Flutter	Clean Architecture - 3 tầng Domain/Data/Presentation
AI hỗ trợ phát triển	Antigravity - tiết kiệm ước tính 30–40% thời gian lập trình

5.2 Hạn chế còn tồn đọng

Mặc dù đạt được những kết quả tích cực, hệ thống LeafGo vẫn còn một số hạn chế cần được ghi nhận. Thứ nhất, tính năng cập nhật vị trí taxi khi ứng dụng chạy ở chế độ nền chưa được tối ưu hóa về mức tiêu thụ pin, do Android ngày càng hạn chế các tiến trình chạy nền. Thứ hai, thuật toán phân công taxi hiển thị tại chưa thông minh - hệ thống phát thông báo đến tất cả taxi trực tuyến thay vì ưu tiên taxi gần nhất, dẫn đến việc xử lý nhiều yêu cầu đồng thời không cần thiết.

Thứ ba, ứng dụng chưa tích hợp hệ thống thanh toán trực tuyến; hiện tại chỉ hỗ trợ thanh toán tiền mặt, đây là hạn chế đáng kể so với các ứng dụng thương mại. Thứ tư, ứng dụng chỉ hỗ trợ giao diện tiếng Việt, chưa có hỗ trợ đa ngôn ngữ. Thứ năm, hệ thống chưa có cơ chế push notification khi ứng dụng bị đóng hoàn toàn (killed state), khiến taxi có thể bỏ lỡ yêu cầu đặt xe khi không mở app.

5.3 Hướng phát triển trong tương lai

Dựa trên những hạn chế đã xác định, nhóm đề xuất các hướng phát triển cho phiên bản tiếp theo của hệ thống LeafGo. Về mặt kỹ thuật, ưu tiên hàng đầu là triển khai thuật toán phân công tài xế thông minh dựa trên tọa độ địa lý, sử dụng truy vấn spatial trong SQL Server để tìm tài xế gần nhất thay vì phát thông báo đại trà. Tiếp theo là tích hợp Firebase Cloud Messaging (FCM) để gửi push notification khi ứng dụng ở trạng thái nền hoặc đã đóng, đảm bảo tài xế không bỏ lỡ chuyển đi.

Về mặt tính năng, hướng phát triển quan trọng là tích hợp cổng thanh toán trực tuyến (VNPay hoặc MoMo) để hỗ trợ thanh toán không dùng tiền mặt, nâng cao trải nghiệm người dùng và đáp ứng xu hướng thanh toán số tại Việt Nam. Ngoài ra, bổ sung tính năng chat realtime giữa khách hàng và tài xế trong suốt chuyến đi, hỗ trợ đa ngôn ngữ (tiếng Anh), và xây dựng thêm ứng dụng web quản trị viên riêng biệt với dashboard phân tích dữ liệu nâng cao là những hướng mở rộng có giá trị thực tiễn cao.

Về mặt hạ tầng, khi mở rộng quy mô người dùng, hệ thống cần được triển khai trên cloud (Azure hoặc AWS), áp dụng kiến trúc microservices để tách biệt các dịch vụ và đảm bảo khả năng mở rộng theo chiều ngang. Cơ chế caching với Redis cho các dữ liệu thường xuyên truy cập (danh sách tài xế trực tuyến, bảng giá cước) cũng cần được xem xét để cải thiện hiệu năng tổng thể của hệ thống.

TÀI LIỆU THAM KHẢO

- [1] Google Developers (2024). *Flutter — Build apps for any screen*, Google LLC, <https://flutter.dev>, truy cập ngày 01/06/2026.
- [2] Google Developers (2024). *Flutter architectural overview*, Google LLC, <https://docs.flutter.dev/resources/architectural-overview>, truy cập ngày 01/05/2026.
- [3] Google Developers (2024). *The Dart programming language*, Google LLC, <https://dart.dev>, truy cập ngày 01/06/2026.
- [4] Martin, R. C. (2012). *The Clean Architecture*, The Clean Coder Blog, <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>, truy cập ngày 01/06/2026.
- [5] Microsoft (2024). *ASP.NET Core documentation*, Microsoft Corporation, <https://learn.microsoft.com/en-us/aspnet/core>, truy cập ngày 01/06/2026.
- [6] Google Firebase (2024). *Firebase Authentication*, Google LLC, <https://firebase.google.com/docs/auth>, truy cập ngày 01/06/2026.
- [7] Jones, M., Bradley, J., and Sakimura, N. (2015). *JSON Web Token (JWT)*, Internet Engineering Task Force (IETF), RFC 7519, <https://datatracker.ietf.org/doc/html/rfc7519>, truy cập ngày 01/06/2026.
- [8] Microsoft (2024). *SQL Server technical documentation*, Microsoft Corporation, <https://learn.microsoft.com/en-us/sql/sql-server>, truy cập ngày 01/06/2026.
- [9] OpenStreetMap Foundation (2024). *OpenStreetMap*, OpenStreetMap Foundation, <https://www.openstreetmap.org>, truy cập ngày 01/06/2026.
- [10] HeiGIT Institute (2024). *OpenRouteService API Documentation*, Heidelberg Institute for Geoinformation Technology, <https://openrouteservice.org/dev/#/api-docs>, truy cập ngày 01/06/2026.

[11] Grab Holdings Inc. (2024). *Grab — Everyday Everything App*, Grab Holdings Inc., <https://www.grab.com/vn>, truy cập ngày 01/06/2026.

[12] Be Group JSC (2024). *Be — Ứng dụng đặt xe thuần Việt*, Be Group JSC, <https://be.com.vn>, truy cập ngày 01/06/2026.